Filtros digitales

Mag. Ing. CIP Moises Meza Rodriguez

Universidad Peruana Cayetano Heredia

Objetivos de la clase

- Comprender el concepto de filtrado digital
- Diferenciar FIR e IIR
- Analizar respuestas en frecuencia
- Comprender el diseño mediante ventanas
- Estudiar familias clásicas IIR
- Comprender biquads
- Entender la transformación bilineal
- Implementar filtros en Python usando SciPy

Motivación

Señales

- ECG
- EEG
- EMG
- Ultrasonido

Procesamiento

- Eliminar ruido
- Restaurar señales
- Separar espectros
- Extraer características

Aplicaciones

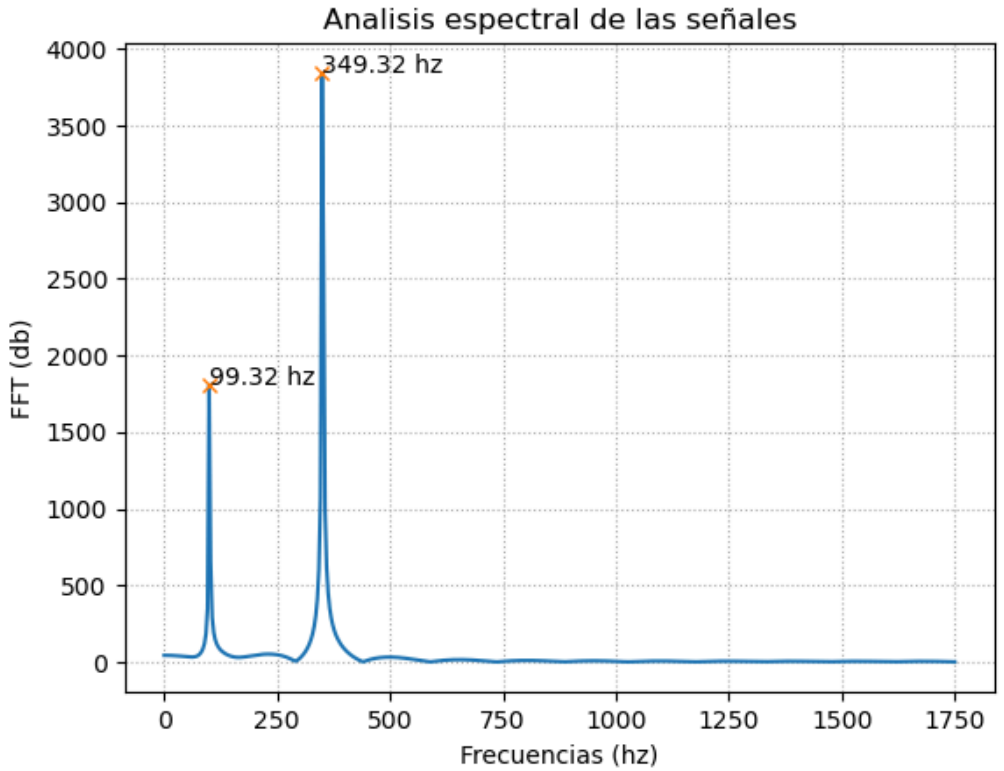
- Control de robots
- Speech decoding
- Rehabilitación
- Apoyo al diagnostico

Concepto de filtro

Un filtro modifica selectivamente componentes frecuenciales.

$$x[n] \rightarrow H(z) \rightarrow y[n]$$

- Señal original: 100hz + 350hz
- Filtro digital: lowpass
- Señal procesada: 100hz

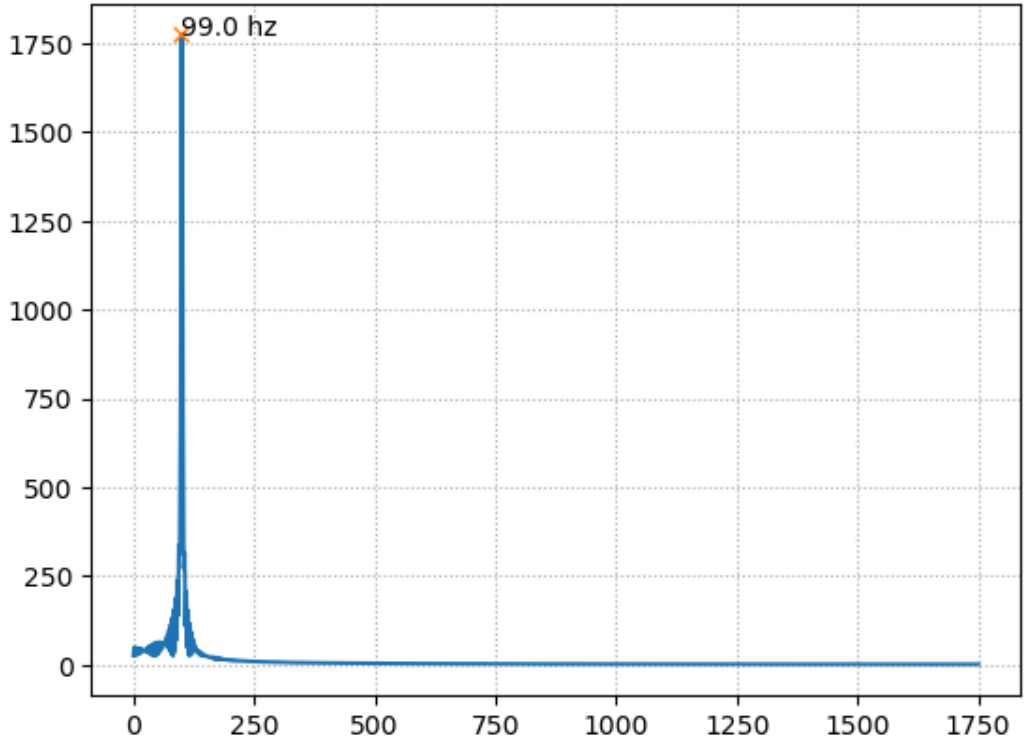


Concepto de filtro

Un filtro modifica selectivamente componentes frecuenciales.

$$x[n] \rightarrow H(z) \rightarrow y[n]$$

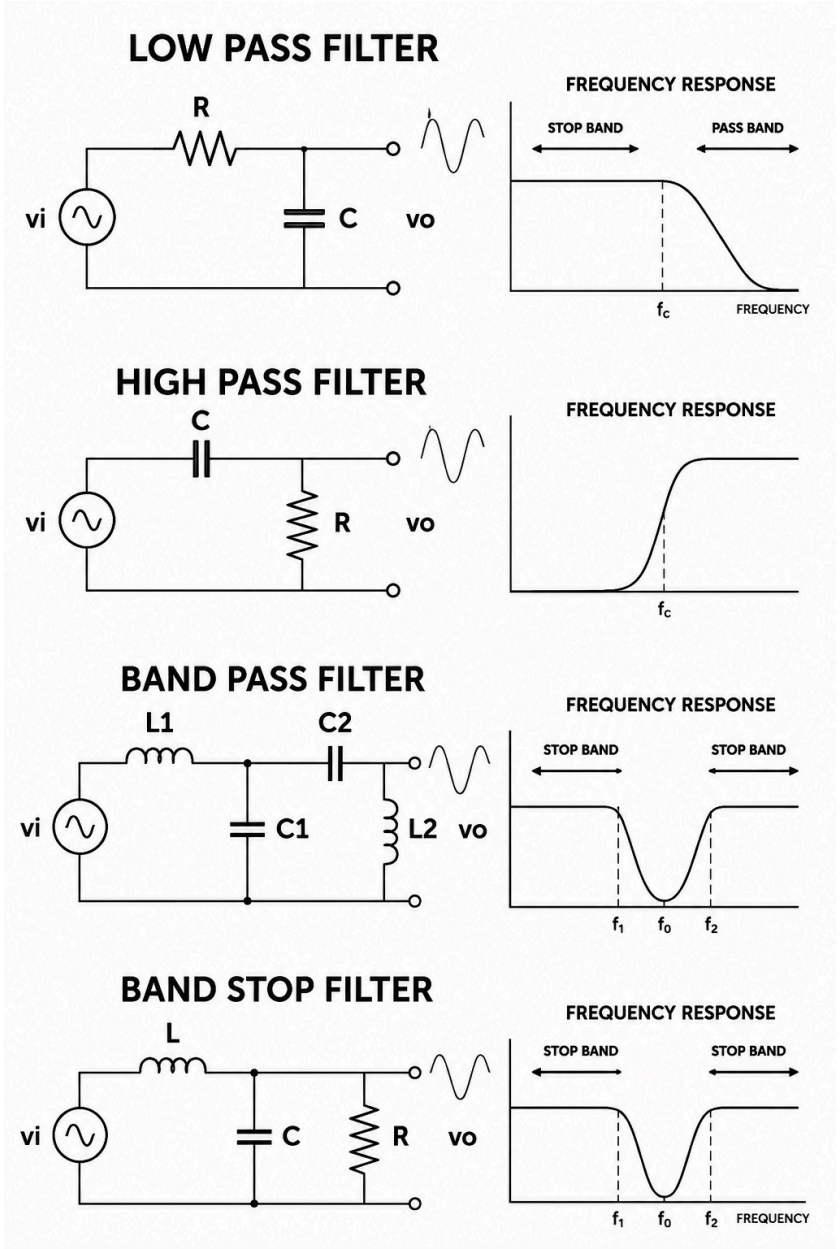
- Señal original: 100hz + 350hz
- Filtro digital: lowpass
- Señal procesada: 100hz



Clasificación de filtros

Según respuesta frecuencial

- Pasa bajas (Low-pass)
- Pasa altas (High-pass)
- Pasa banda (Band-pass)
- Rechaza banda / Notch



Filtro pasa bajas

Circuito RC la salida se toma sobre el capacitor.

Impedancias

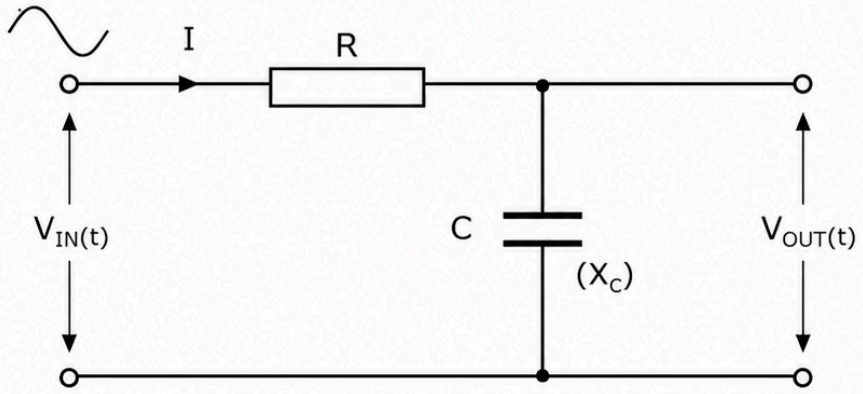
$$Z_R = R$$

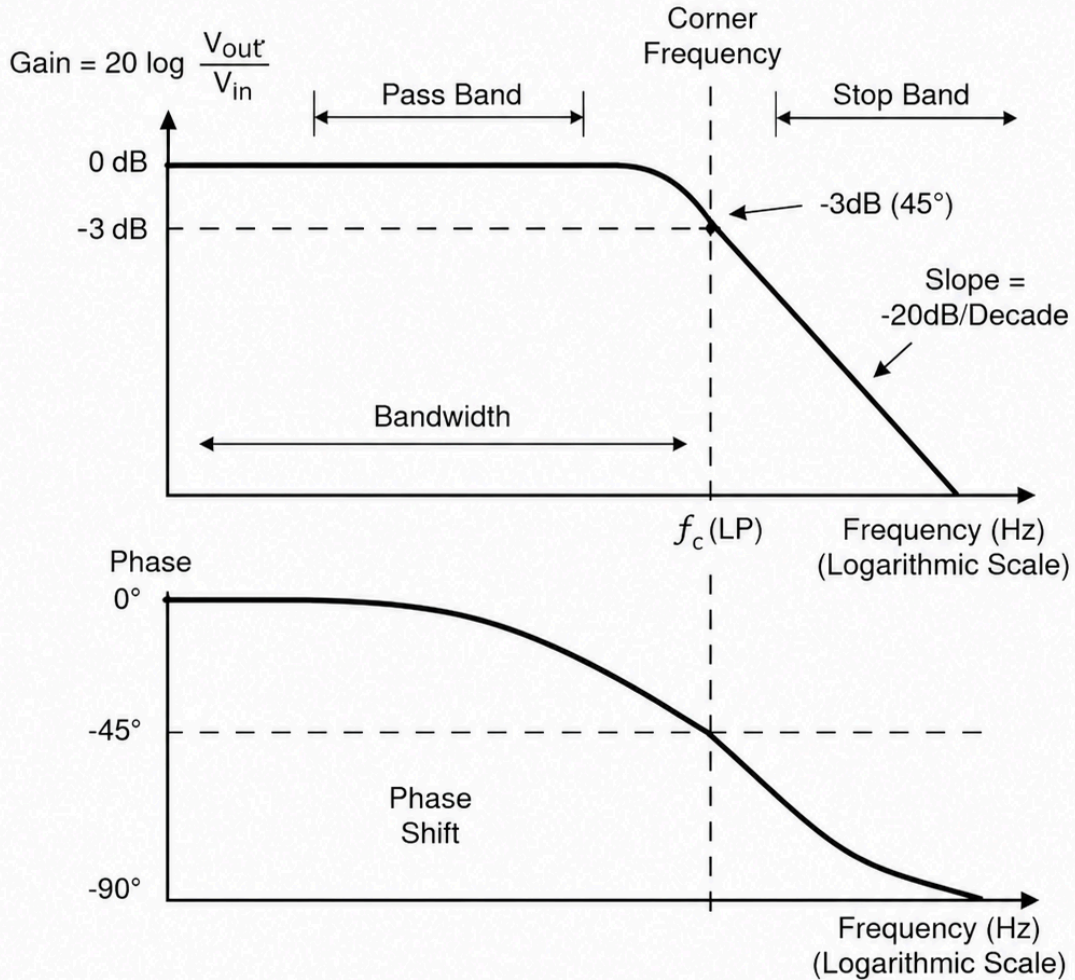
$$Z_C = \frac{1}{sC}$$

Divisor de voltaje

$$H(s) = \frac{V_{out}(s)}{V_{in}(s)}$$

$$H(s) = \frac{Z_C}{R + Z_C}$$





Función de transferencia

Simplificación

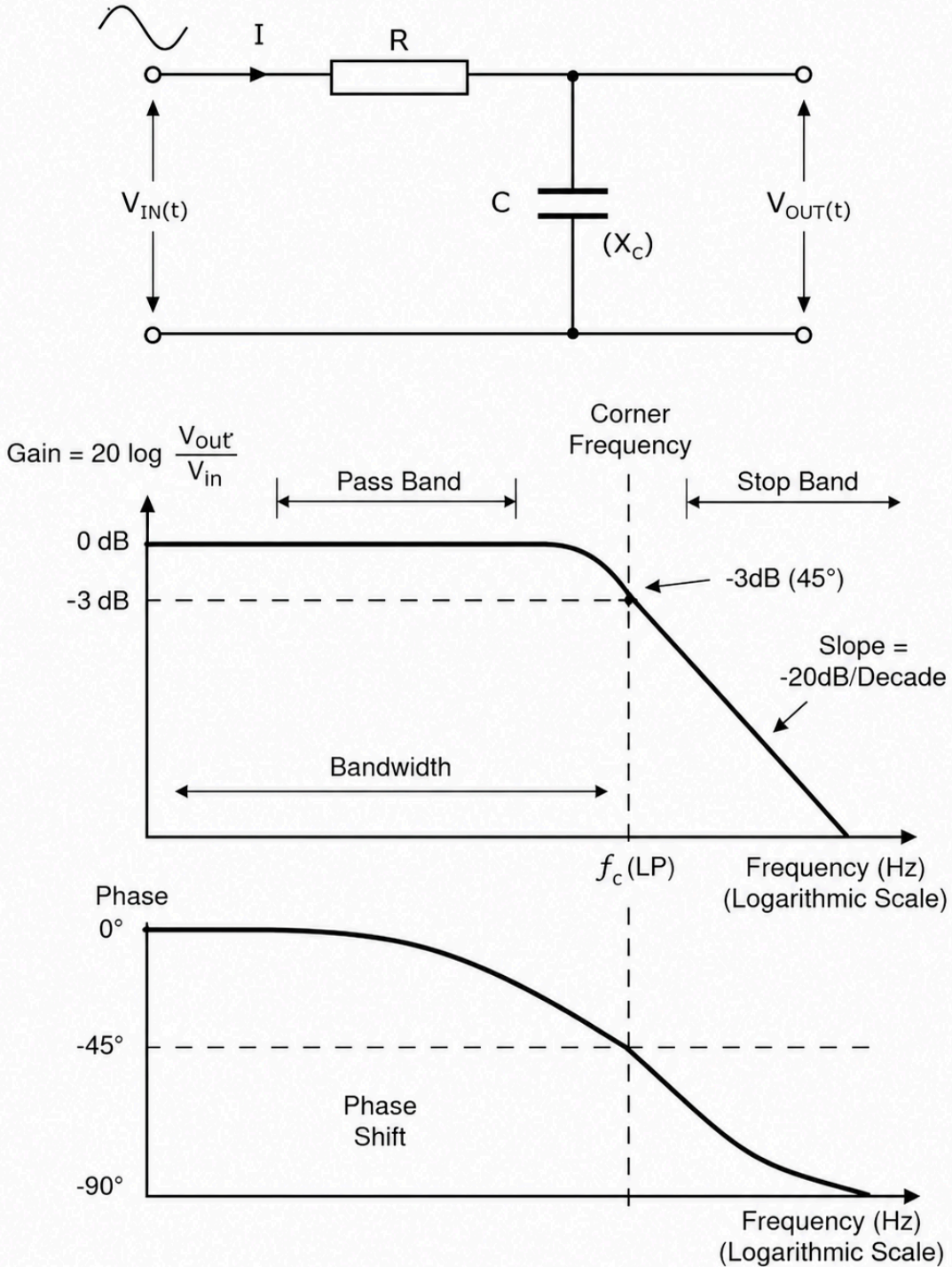
$$H(s) = \frac{\frac{1}{sC}}{R + \frac{1}{sC}}$$

$$\boxed{H(s) = \frac{1}{1 + sRC}}$$

Frecuencia de corte

$$\omega_c = \frac{1}{RC}$$

$$f_c = \frac{1}{2\pi RC}$$



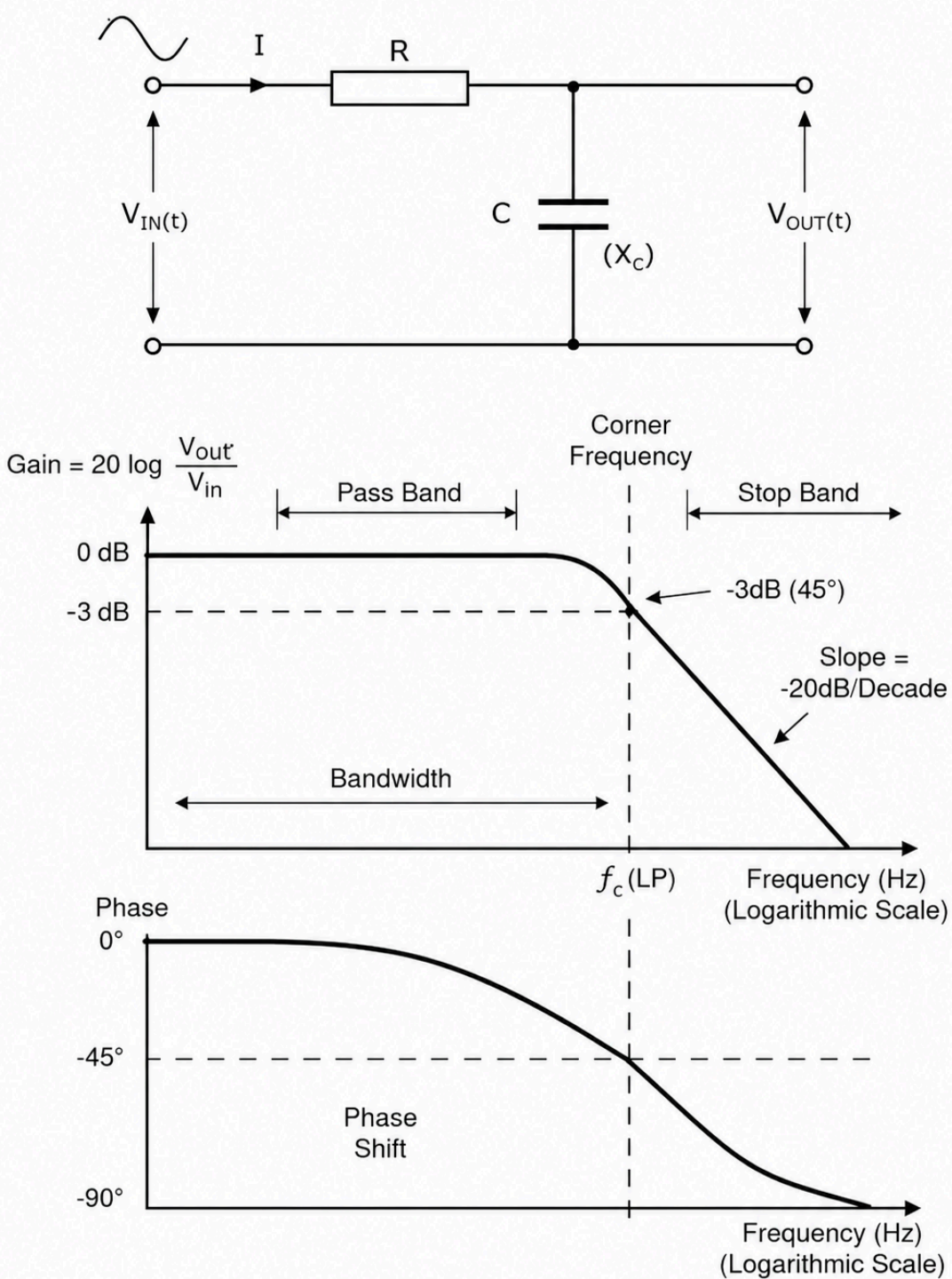
Respuesta en frecuencia

Magnitud

$$|H(j\omega)| = \frac{1}{\sqrt{1 + (\omega RC)^2}}$$

Fase

$$\angle H(j\omega) = -\tan^{-1}(\omega RC)$$



Código en python Low pass

```
from scipy import signal
import numpy as np
import matplotlib.pyplot as plt
```

```
fc=50
wc=2*np.pi*fc

num=wc
den=[1, wc]

h_s=signal.TransferFunction(num,den)  #lti
```

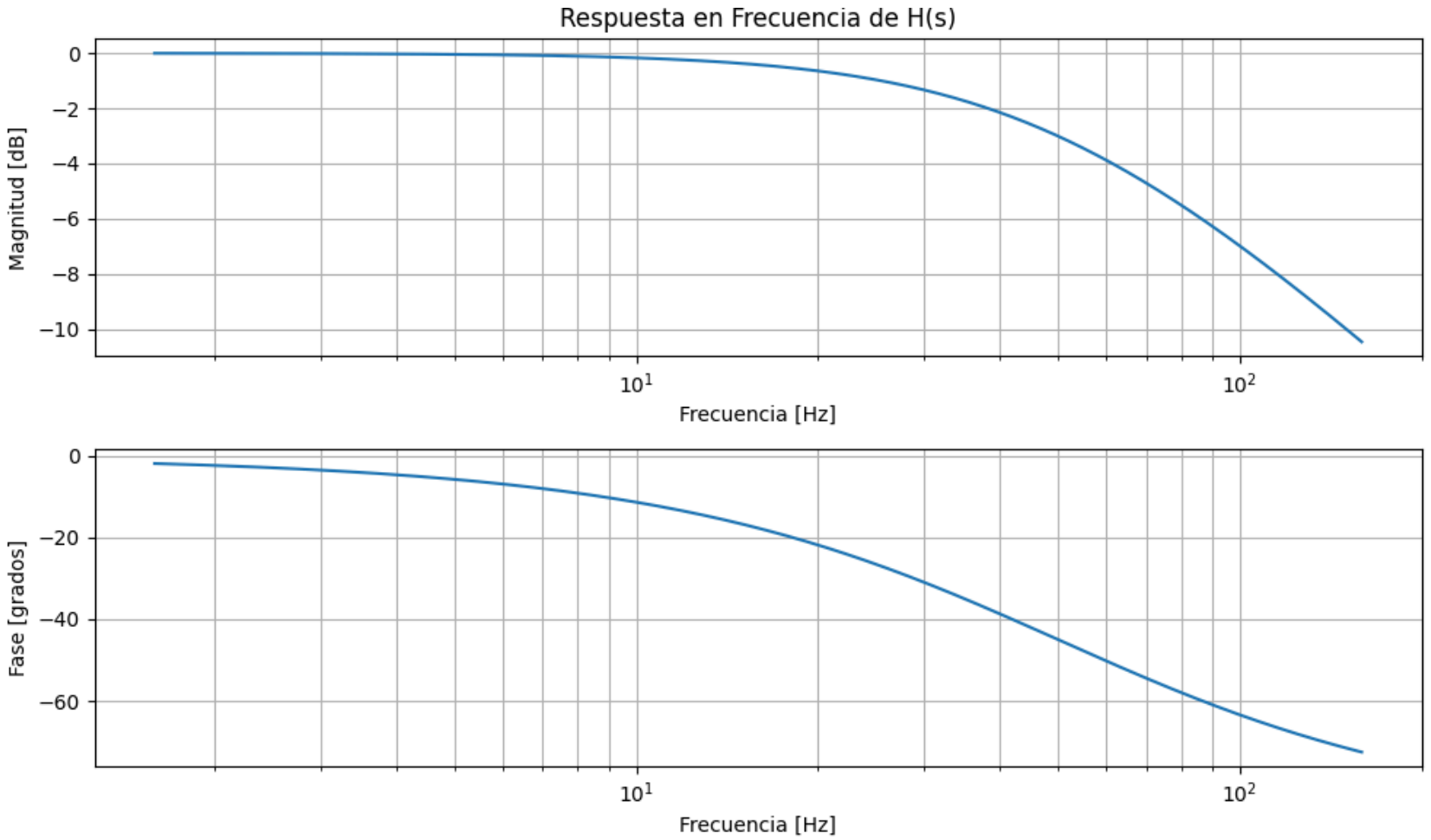
Código en python respuesta en frecuencia

```
w, H = signal.freqresp(h_s)    # w, mag, phase = signal.bode(h_s)
mag, phase = np.abs(H), np.angle(H)
```

```
plt.figure(figsize=(10, 6))
plt.subplot(2, 1, 1)
plt.semilogx(w / (2 * np.pi), 20 * np.log10(mag))
plt.xlabel('Frecuencia [Hz]')
plt.ylabel('Magnitud [dB]')
plt.title('Respuesta en Frecuencia de H(s)')
plt.grid(True, which='both')

plt.subplot(2, 1, 2)
plt.semilogx(w / (2 * np.pi), np.degrees(phase))
plt.xlabel('Frecuencia [Hz]')
plt.ylabel('Fase [grados]')
plt.grid(True, which='both')
plt.tight_layout()
plt.show()
```

Respues en frecuencia filtro pasa bajas



Filtro pasa bajos discreto

Transformada Bilineal $H_{(s)} \rightarrow H_{(z)}$

$$s = \frac{2}{T}\frac{1 - z^{-1}}{1 + z^{-1}}$$

donde:

- $T \rightarrow$ período de muestreo
- $f_s = \frac{1}{T} \rightarrow$ frecuencia de muestreo

```
fs=1e3 # 1 khz
ts =1/fs

h_d = h_s.to_discrete(ts, method="bilinear")
```

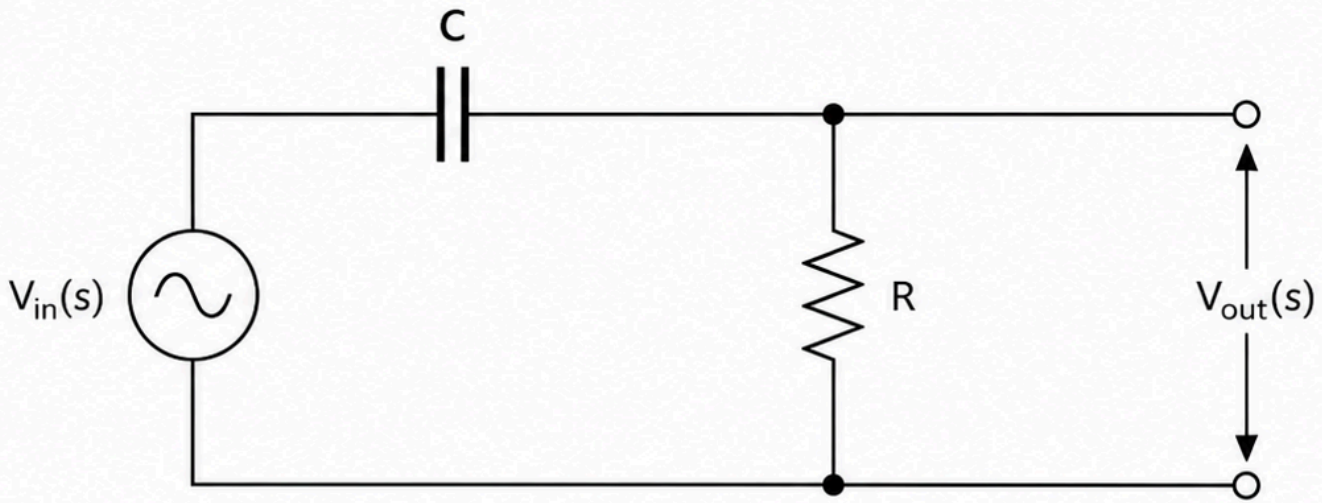
```
a=-h_d.den[1:]
b=h_d.num
```

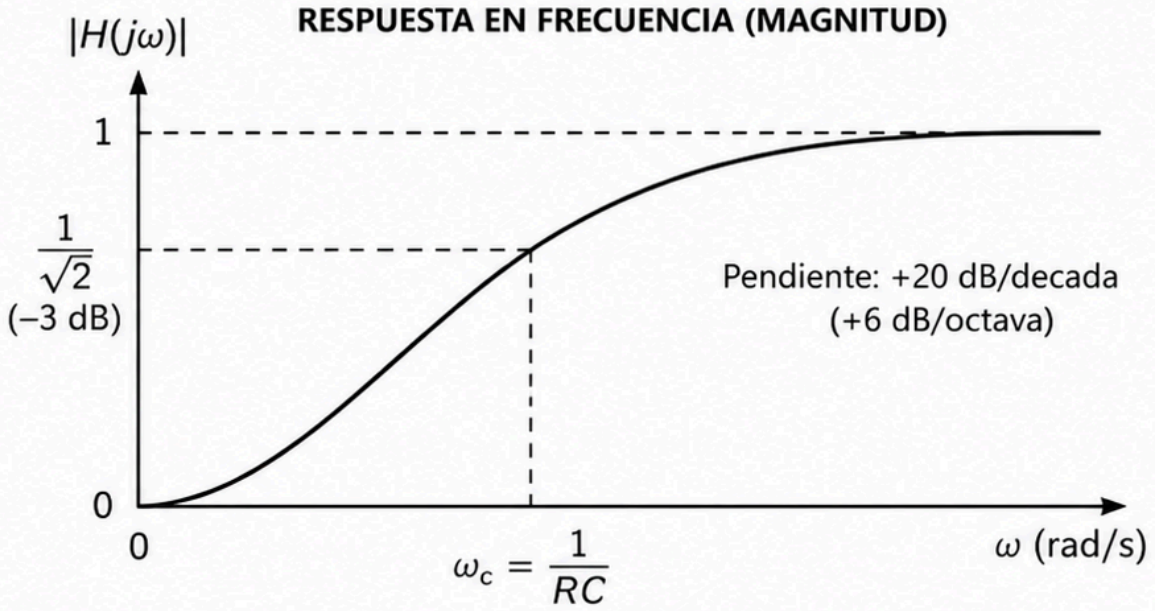
Código completo

https://github.com/MSMRo/DSP_ISB/blob/main/NOTEBOOKS/2_systems/filtro_pasabajo.ipynb



Filtro pasa altas





FUNCIÓN DE TRANSFERENCIA

$$H(s) = \frac{V_{out}(s)}{V_{in}(s)} = \frac{sRC}{1 + sRC}$$

Frecuencia de corte:

$$\omega_c = \frac{1}{RC}$$

$$f_c = \frac{1}{2\pi RC}$$

Filtro pasa altas

Función de transferencia

$$H(s) = \frac{sRC}{1 + sRC} \qquad f_c = \frac{1}{2\pi RC}$$

Elimina bajas frecuencias y deja pasar altas.

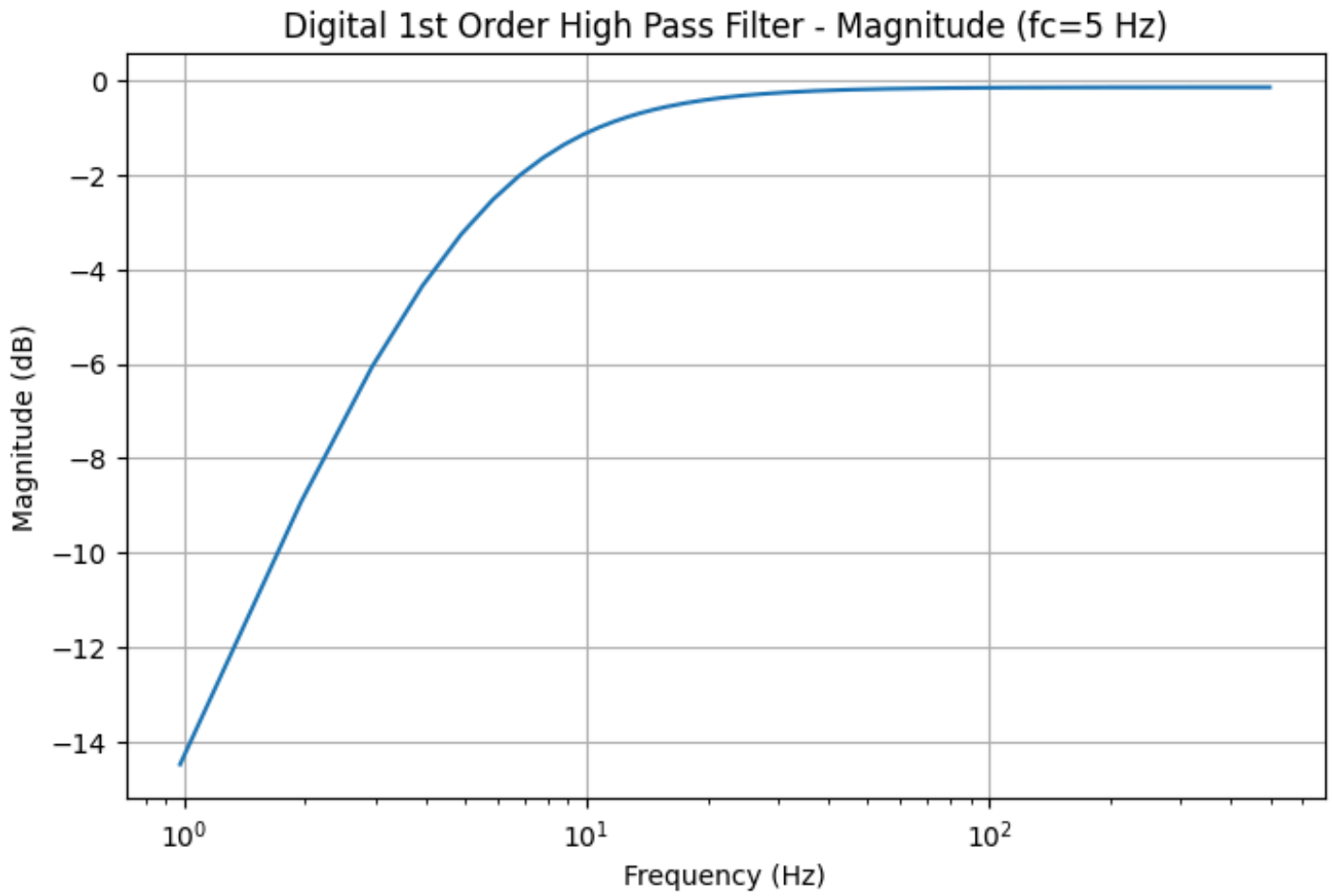
```
fc = 5                    # R = 1000       # 1 kΩ
RC = 1/(2*np.pi*fc)       # C = 1e-6       # 1 µF

num = [RC, 0]
den = [RC, 1]

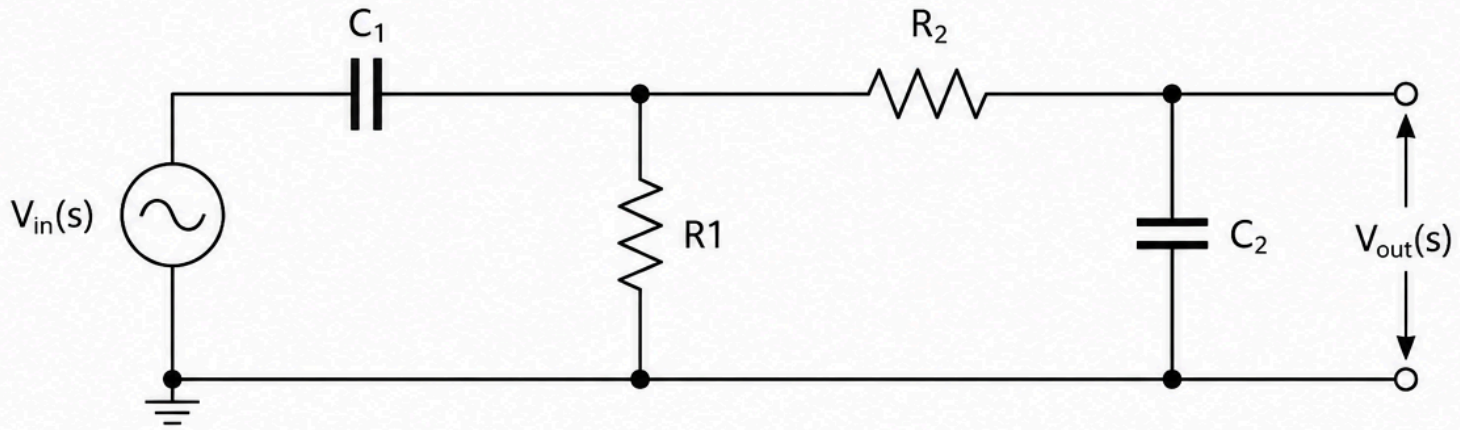
system = signal.TransferFunction(num, den)
w, mag, phase = signal.bode(system)

plt.semilogx(w/(2*np.pi), mag)
plt.title("RC High Pass - Magnitude")
plt.grid()
plt.show()
```

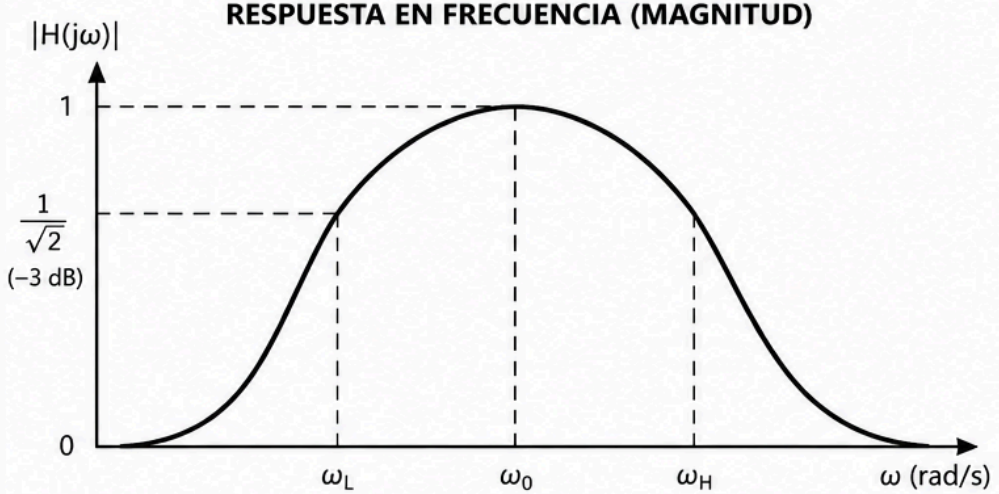
Respuesta en frecuencia



Filtro pasabanda



RESPUESTA EN FRECUENCIA (MAGNITUD)



$$\omega_L = \frac{1}{R_1 C_1} \qquad \omega_0 = \frac{1}{\sqrt{R_1 R_2 C_1 C_2}} \qquad \omega_H = \frac{1}{R_2 C_2}$$

Ancho de banda: $B = \omega_H - \omega_L$

FUNCIÓN DE TRANSFERENCIA

$$H(s) = \frac{V_{out}(s)}{V_{in}(s)} = \frac{sR_1C_1}{(1+sR_1C_1)(1+sR_2C_2)}$$

PARÁMETROS PRINCIPALES

- Frecuencia central (resonancia):
$$\omega_0 = \frac{1}{\sqrt{R_1 R_2 C_1 C_2}}$$
- Frecuencias de corte (−3 dB):
$$\omega_L = \frac{1}{R_1 C_1}, \quad \omega_H = \frac{1}{R_2 C_2}$$
- Ancho de banda: $B = \omega_H - \omega_L$
- Factor de calidad: $Q = \frac{\omega_0}{B}$

Función de transferencia

$$H(s) = \frac{sR_1C_1}{1 + sR_1C_1 + s^2R_2C_2C} \qquad f_c = \frac{1}{2\pi RC}$$

Elimina una banda específica.

```
fc_hp = 5 # High-pass cutoff frequency
fc_lp = 50 # Low-pass cutoff frequency

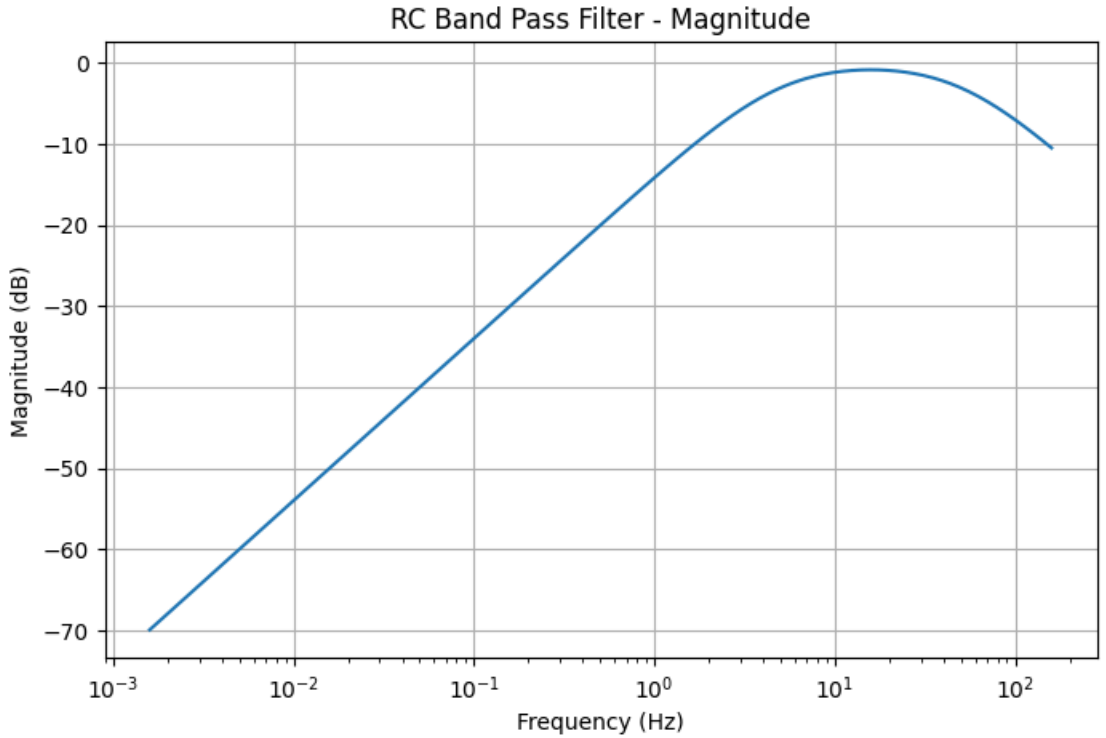
RC_hp = 1 / (2 * np.pi * fc_hp)
RC_lp = 1 / (2 * np.pi * fc_lp)

# Numerator coefficients for H(s) = (RC_hp * s)
num = [RC_hp, 0]

# Denominator coefficients for H(s) = (RC_hp * RC_lp * s^2 + (RC_hp + RC_lp) * s + 1)
den = [RC_hp * RC_lp, RC_hp + RC_lp, 1]
```

```
system_bp = signal.TransferFunction(num, den)
w_bp, mag_bp, phase_bp = signal.bode(system_bp)

plt.figure(figsize=(8,5))
plt.semilogx(w_bp/(2*np.pi), mag_bp)
plt.title("RC Band Pass Filter - Magnitude")
plt.xlabel("Frequency (Hz)")
plt.ylabel("Magnitude (dB)")
plt.grid(True)
plt.show()
```



Analógicos vs Digitales

Analógicos	Digitales
Continuos	Discretos
RLC / Opamps	DSP / MCU
Ecuaciones diferenciales	Ecuaciones en diferencias
Sensibles a temperatura	Reproducibles
Difícil reconfiguración	Reprogramables

Convolución

Base del filtrado

$$y[n] = x[n] * h[n]$$

$$y[n] = \sum_{k=-\infty}^{\infty} x[k]h[n-k]$$

Respuesta al impulso

Si:

$$x[n] = \delta[n]$$

Entonces:

$$y[n] = h[n]$$

Importancia

Caracteriza completamente al sistema LTI.

Función de transferencia

$$H(z) = \frac{Y(z)}{X(z)}$$

$$H(z) = \frac{b_0 + b_1 z^{-1} + \cdots + b_M z^{-M}}{1 + a_1 z^{-1} + \cdots + a_N z^{-N}}$$

Polos y ceros

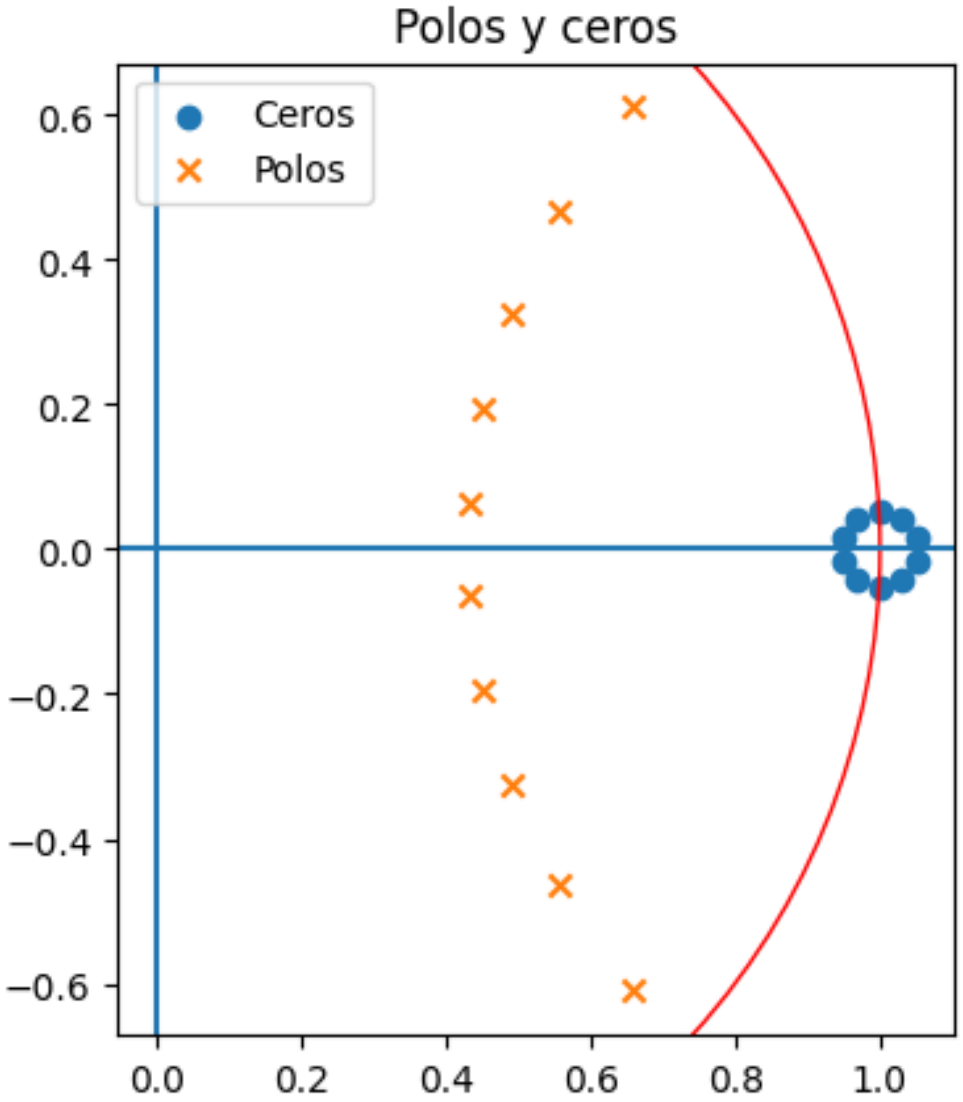
```
z, p, k = signal.tf2zpk(b, a)
```

```
plt.scatter(np.real(z), np.imag(z), marker='o', label='Ceros')
plt.scatter(np.real(p), np.imag(p), marker='x', label='Polos')

circle = plt.Circle((0,0), 1, color='r', fill=False)
plt.gca().add_artist(circle)

plt.axhline(0)
plt.axvline(0)
plt.gca().set_aspect('equal', 'box')
plt.legend()
plt.title('Polos y ceros')
plt.show()
```

- Ceros: Atenúan frecuencias
- Polos: Amplifican / generan resonancia
- Estabilidad: Todos los polos deben estar dentro del círculo unidad.



Especificaciones típicas

$\delta_p, \delta_s, \omega_p, \omega_s$

Parámetros

- Banda pasante
- Banda rechazo
- Ripple
- Atenuación
- Banda transición

Interpretación

- ω_p → frecuencia pasante
- ω_s → frecuencia rechazo
- δ_p → ripple pasante
- δ_s → atenuación rechazo

Consideraciones

- menos de -20db se considera atenuado.
- $f_N = \frac{f}{f_s}$, La frecuencia normalizada normalmente se expresa en: $0 \leq fn \leq 0.5$
- $f_{Nyquist} = fs/2$; útil en la fft.

Filtros FIR

Finite Impulse Response

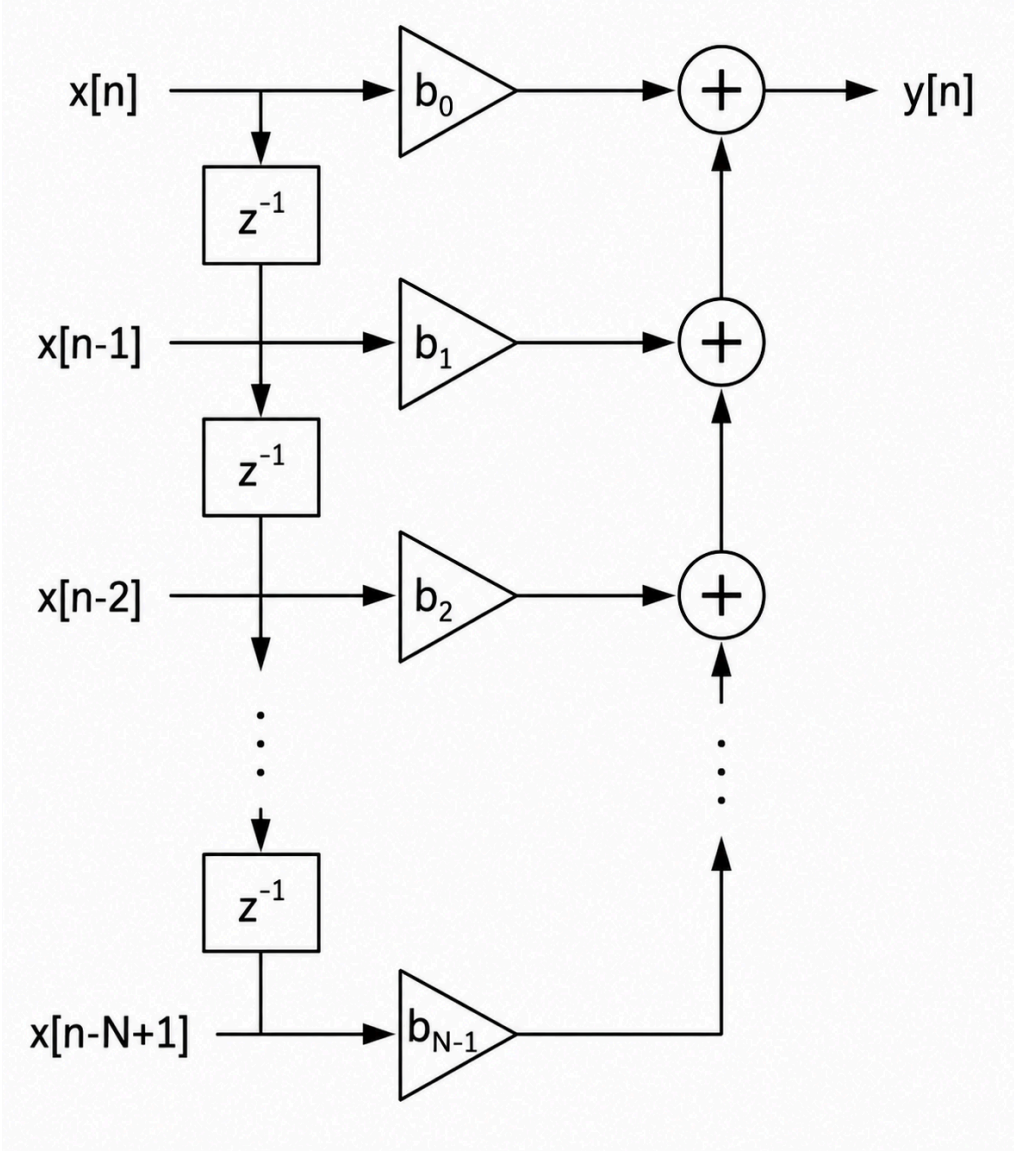
$$y[n] = \sum_{k=0}^{M} b_k x[n-k]$$

Características:

- No recursivos
- Solo ceros
- Fase lineal
- Siempre estables

Interpretación

Promedio ponderado del pasado.



Estructura FIR

Componentes

- Delays
- Multiplicadores
- Sumadores

Métodos de diseño FIR

- Ventanas
- Muestreo en frecuencia
- Remez (óptimo)

Desventajas FIR

- Orden alto
- Mayor memoria
- Mayor costo computacional

Importancia

- sistemas embebidos.
- Equipos biomédicos

Ventajas FIR

- Estabilidad garantizada
- Fase lineal
- Robustos numéricamente
- Implementación sencilla

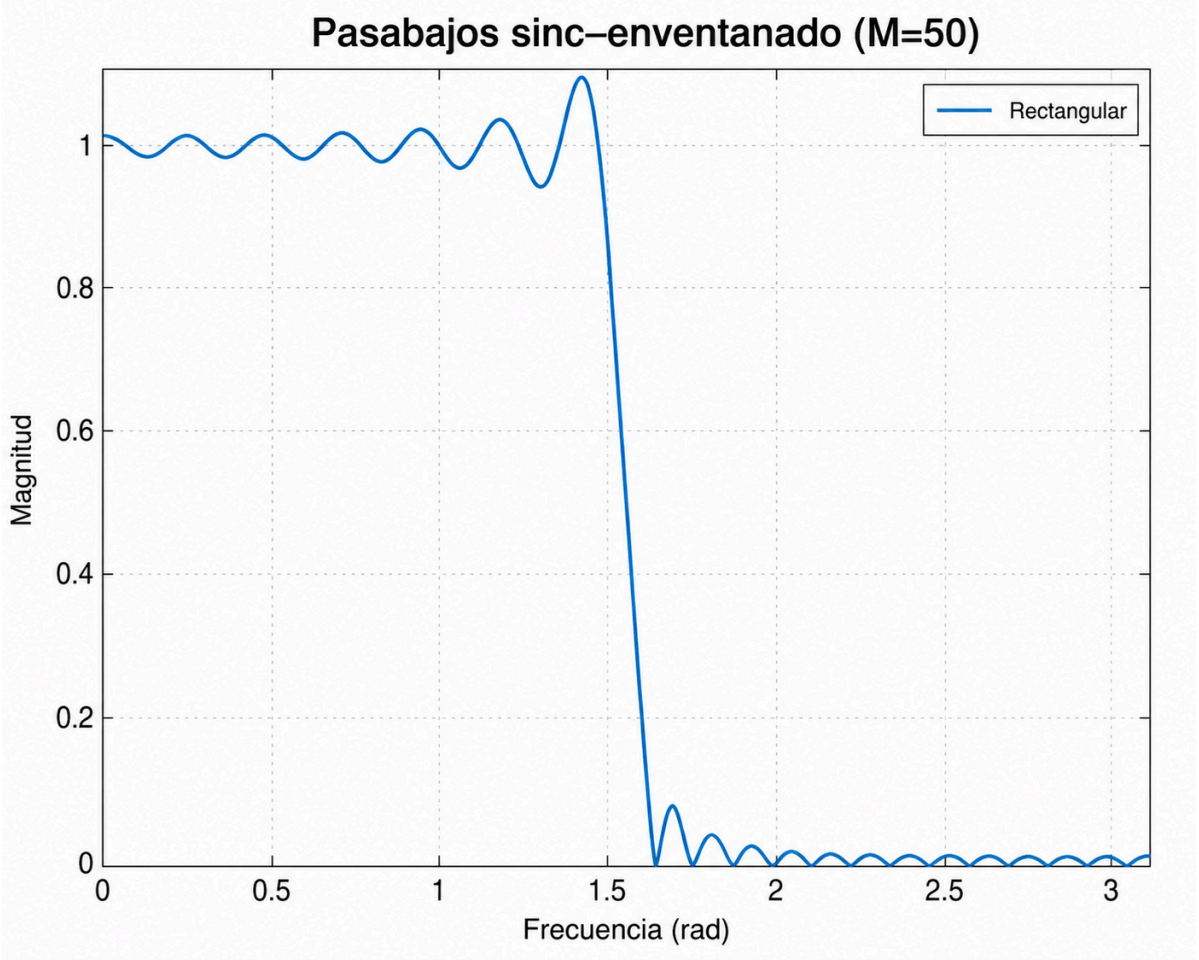
Fenómeno de Gibbs

Problema del truncamiento

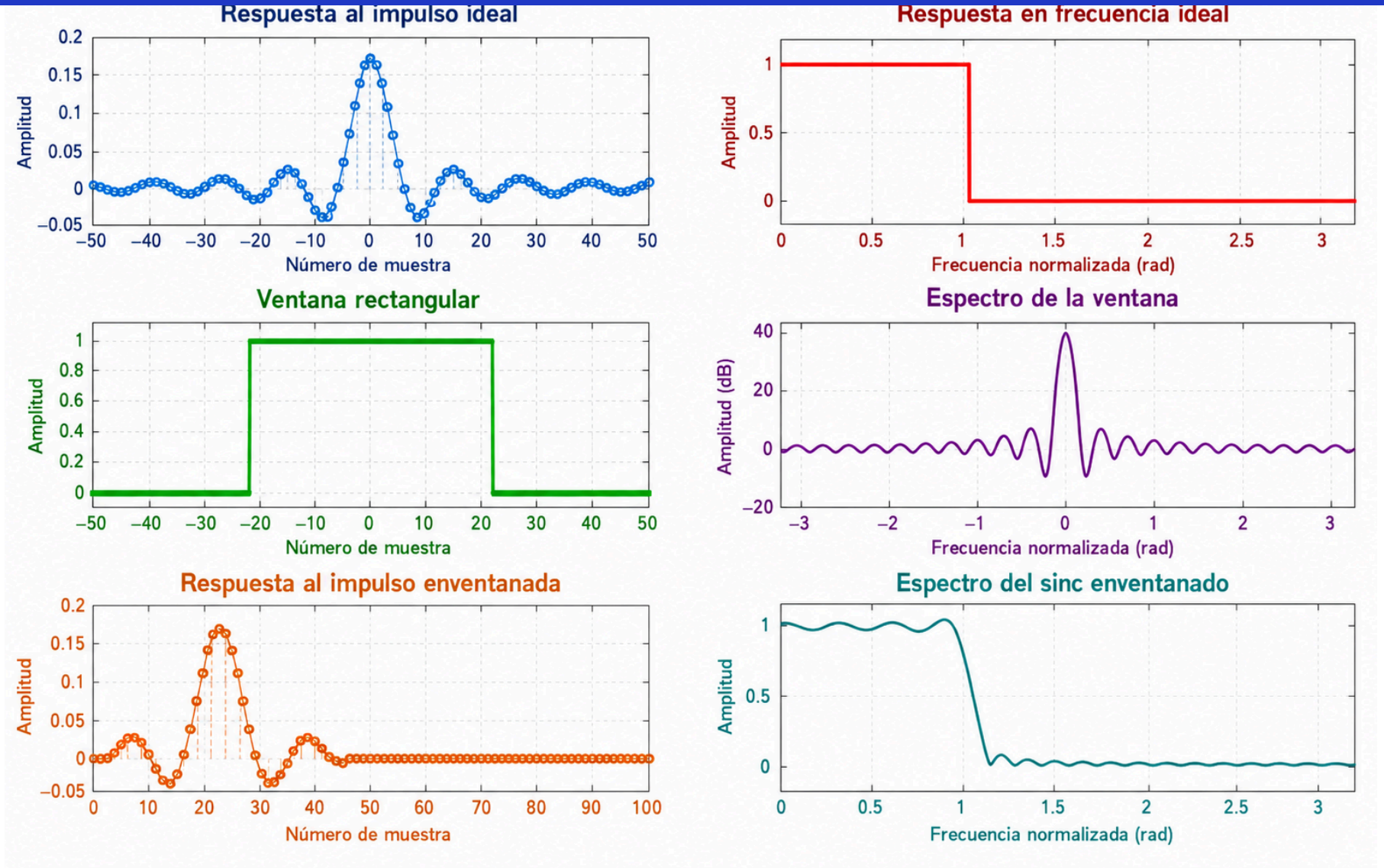
Aparecen oscilaciones cerca del corte.

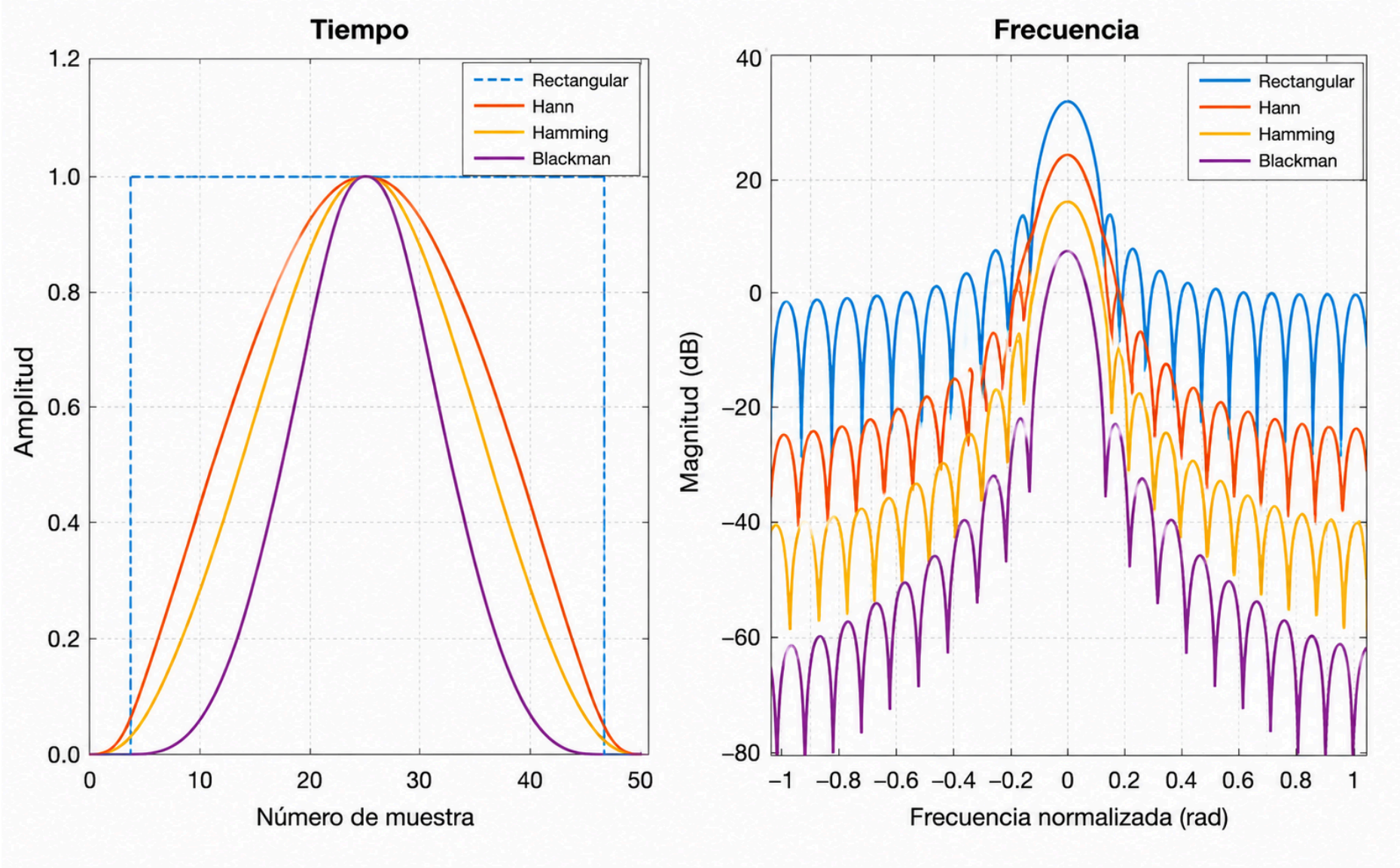
Consecuencias:

- Ripple
- Distorsión espectral



Método de ventanas




Tiempo
Rectangular
Hann
Hamming
Blackman
Amplitud
Número de muestra
Frecuencia
Magnitud (dB)
Frecuencia normalizada (rad)

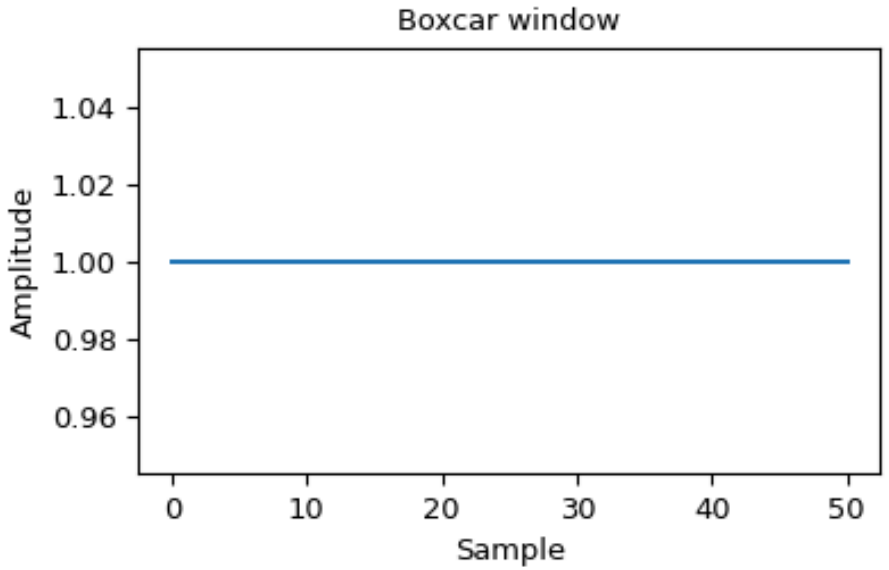
Ventana rectangular

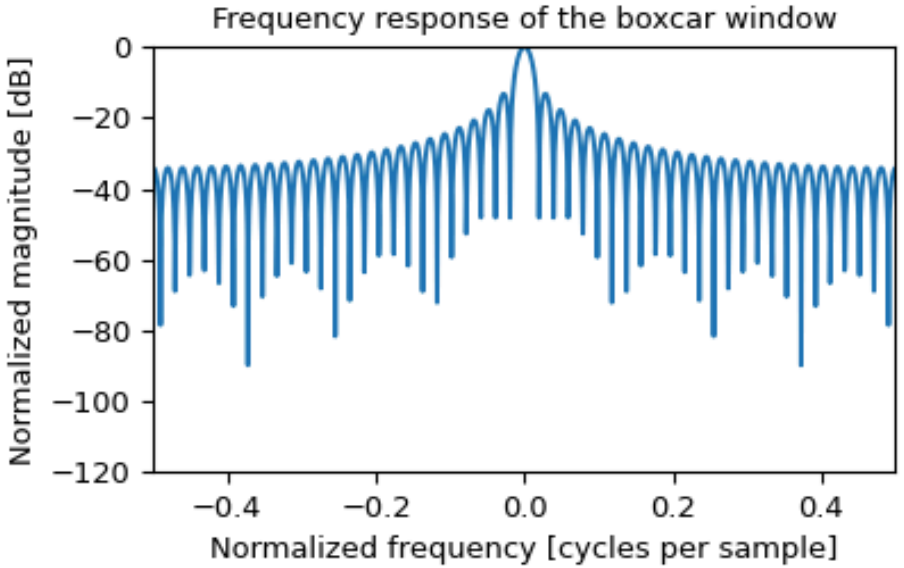
Características

- Mainlobe estrecho
- Poor sidelobe attenuation
- Ripple elevado

```
b = firwin(numtaps, # impar
        cutoff,  # frecuencia de corte
        window='rec',
        pass_zero="lowpass",
        fs=fs)
```

```
y = lfilter(b,[1],x)
y = filtfilt(b,[1],x)
```





Ventana Hann

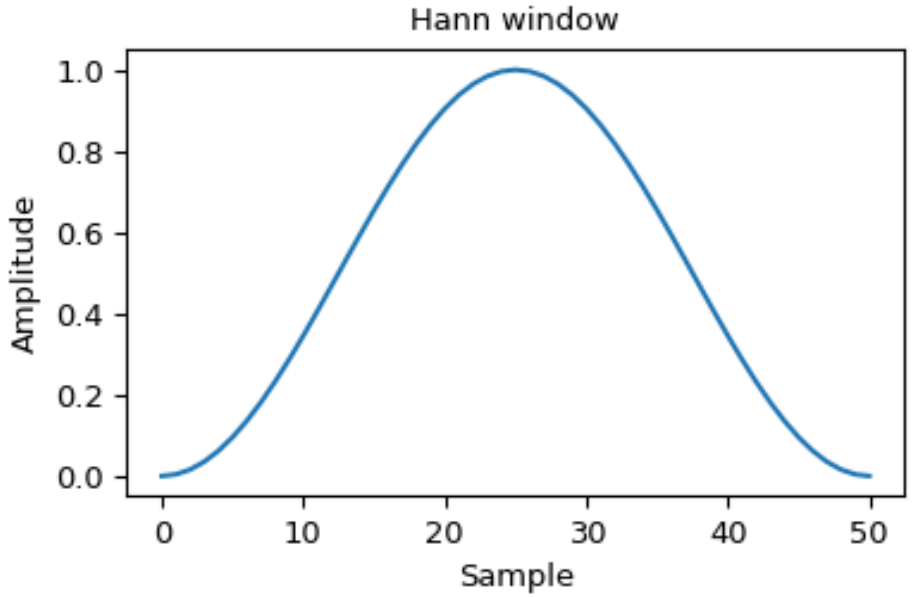
$$w[n] = 0.5 - 0.5\cos\left(\frac{2\pi n}{N}\right)$$

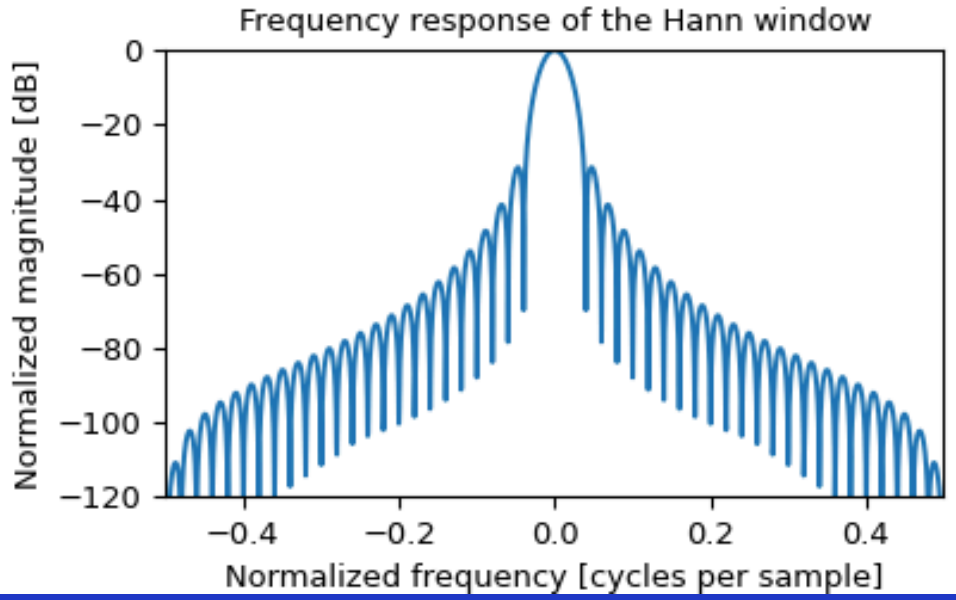
Características

- Buen suavizado
- Ripple moderado

```
b = firwin(numtaps, # impar
        cutoff,  # frecuencia de corte
        window='hann',
        pass_zero="lowpass",
        fs=fs)
```

```
y = lfilter(b,[1],x)
y = filtfilt(b,[1],x)
```





Ventana Hamming

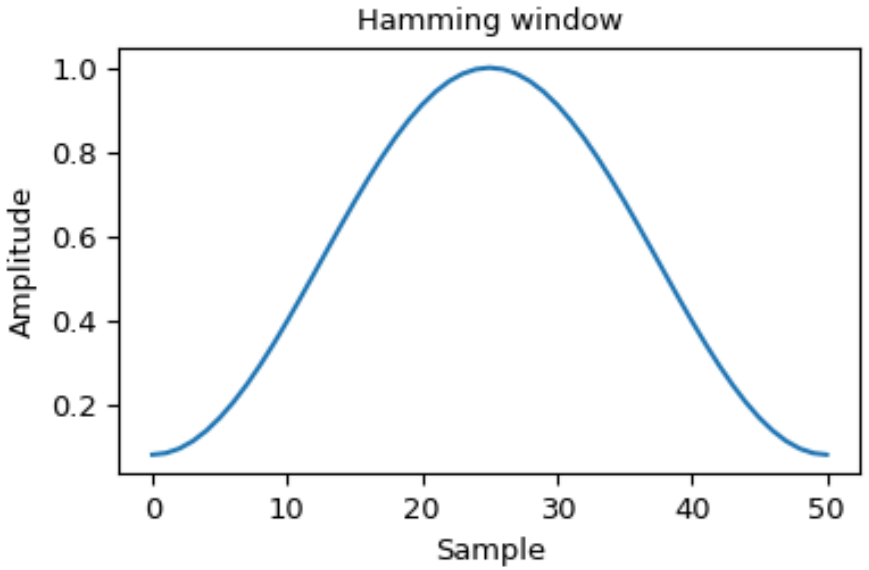
$$w[n] = 0.54 - 0.46 \cos\left(\frac{2\pi n}{N}\right)$$

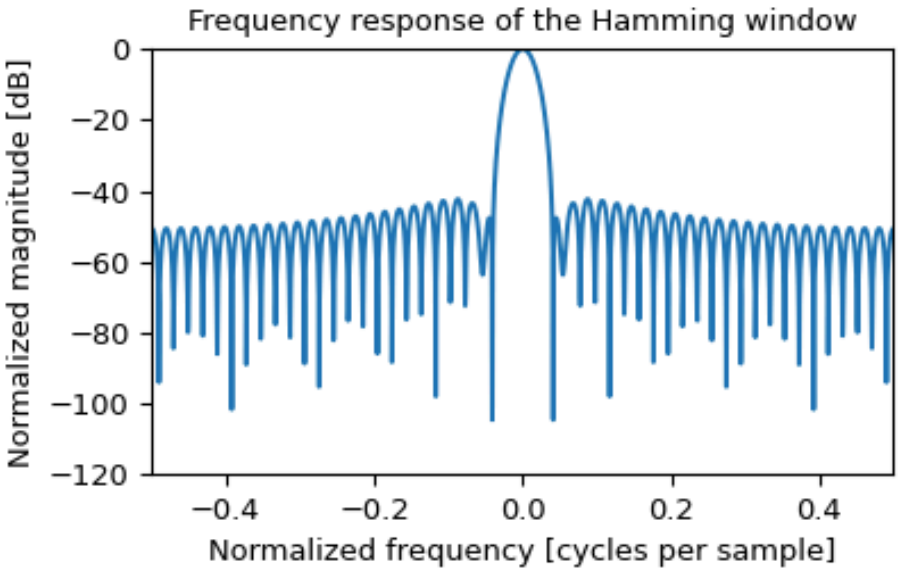
Características

- Muy usada
- Mejor rechazo lateral

```
b = firwin(numtaps, # impar
        cutoff,  # frecuencia de corte
        window='hamming',
        pass_zero="lowpass",
        fs=fs)
```

```
y = lfilter(b,[1],x)
y = filtfilt(b,[1],x)
```





Ventana Blackman

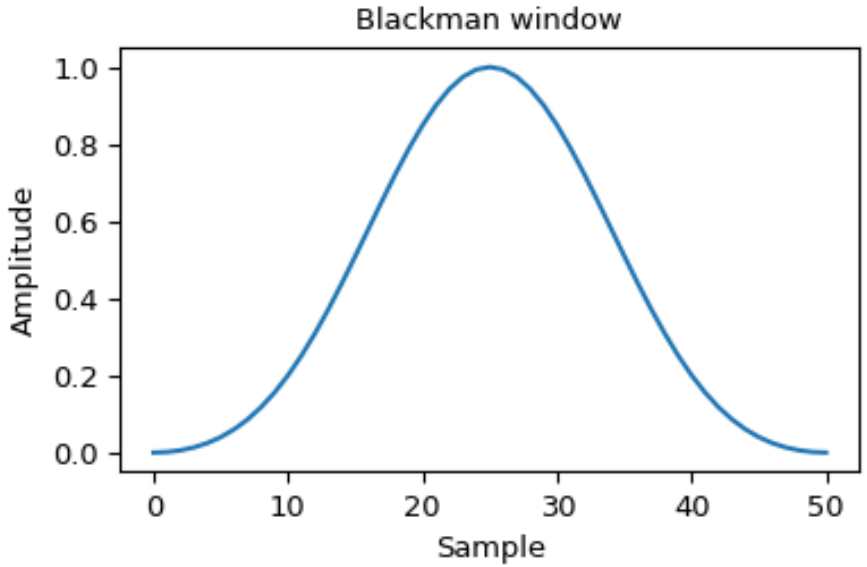
$$w[n] = 0.42 - 0.5\cos\left(\frac{}{N}\right) + 0.08\cos\left(\frac{}{N}\right)$$

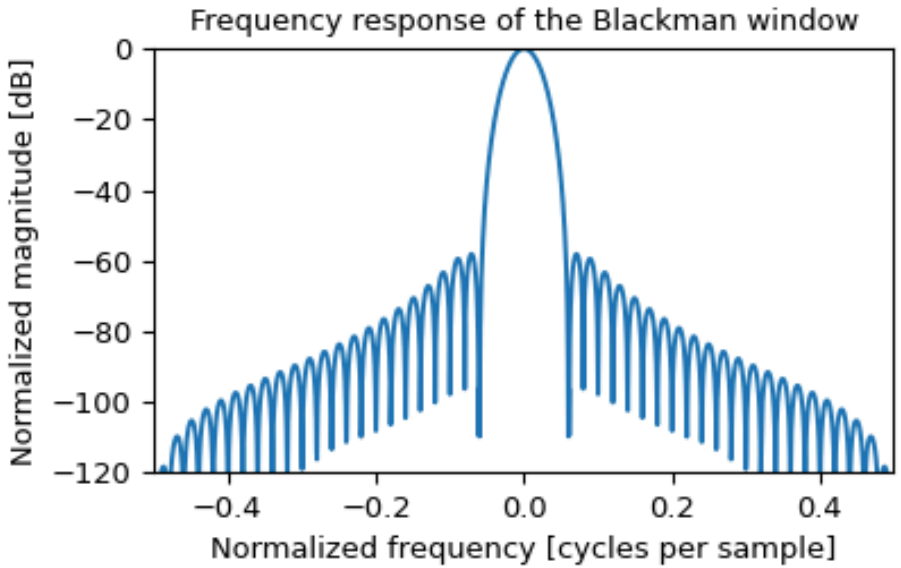
Características

- Excelente atenuación
- Transición más ancha
- Transición

```
b = firwin(numtaps, # impar
        cutoff,  # frecuencia de corte
        window='blackman',
        pass_zero="lowpass",
        fs=fs)
```

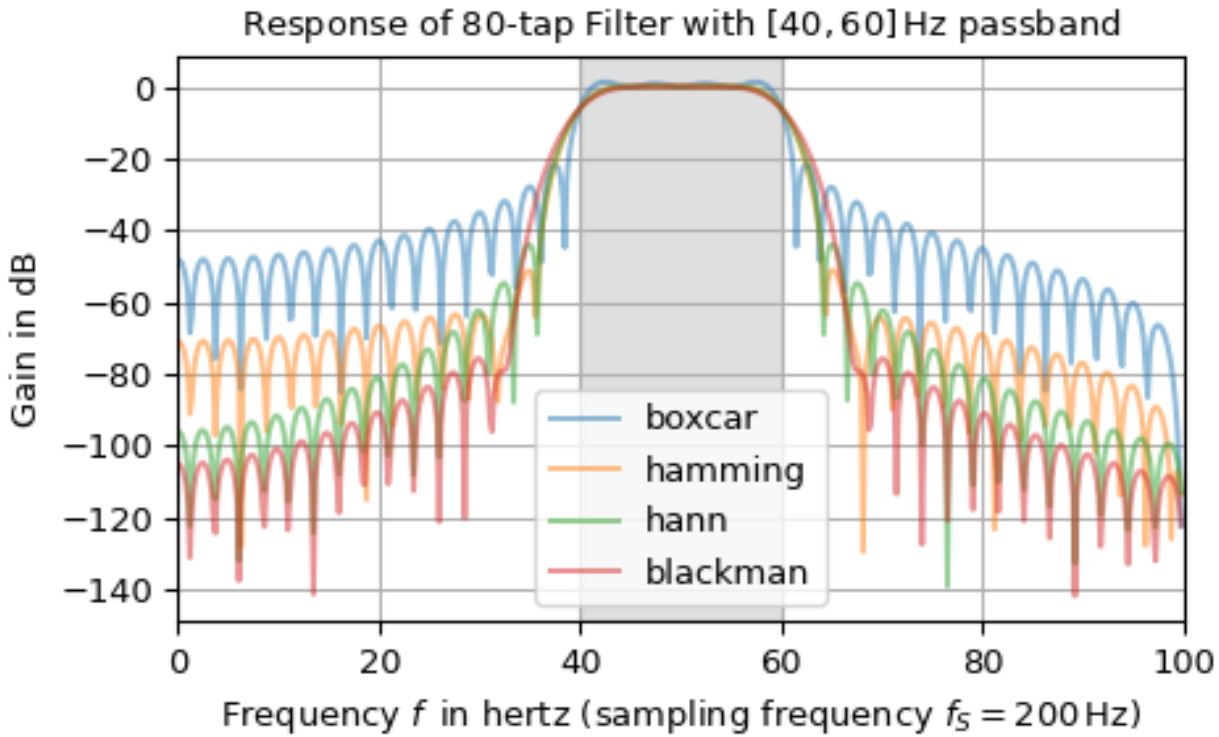
```
y = lfilter(b,[1],x)
y = filtfilt(b,[1],x)
```



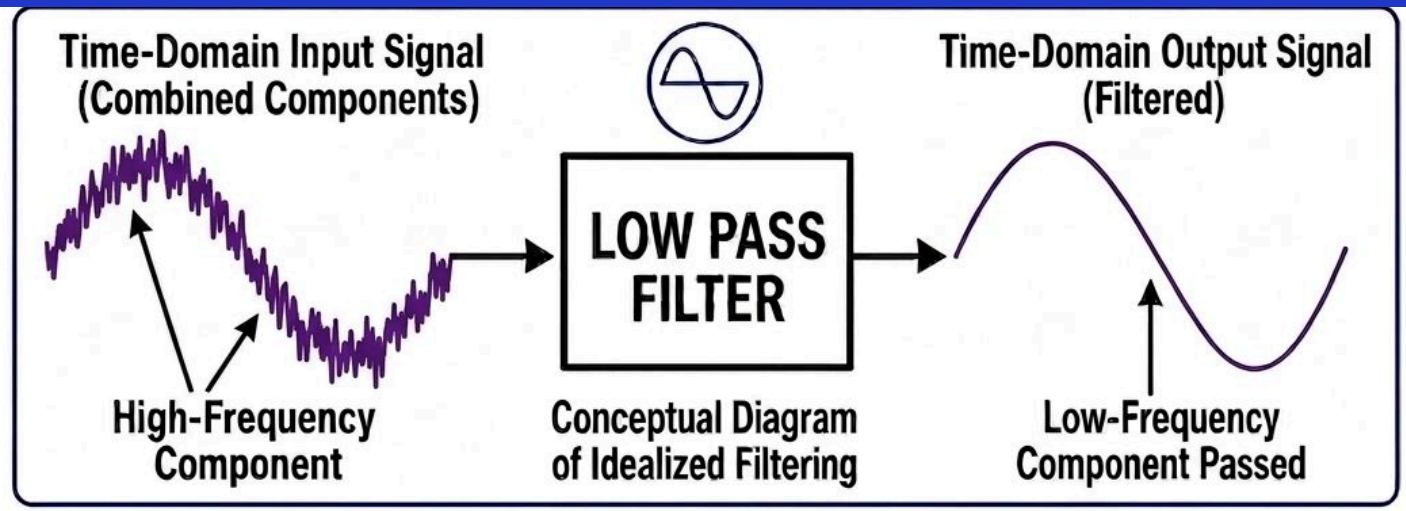


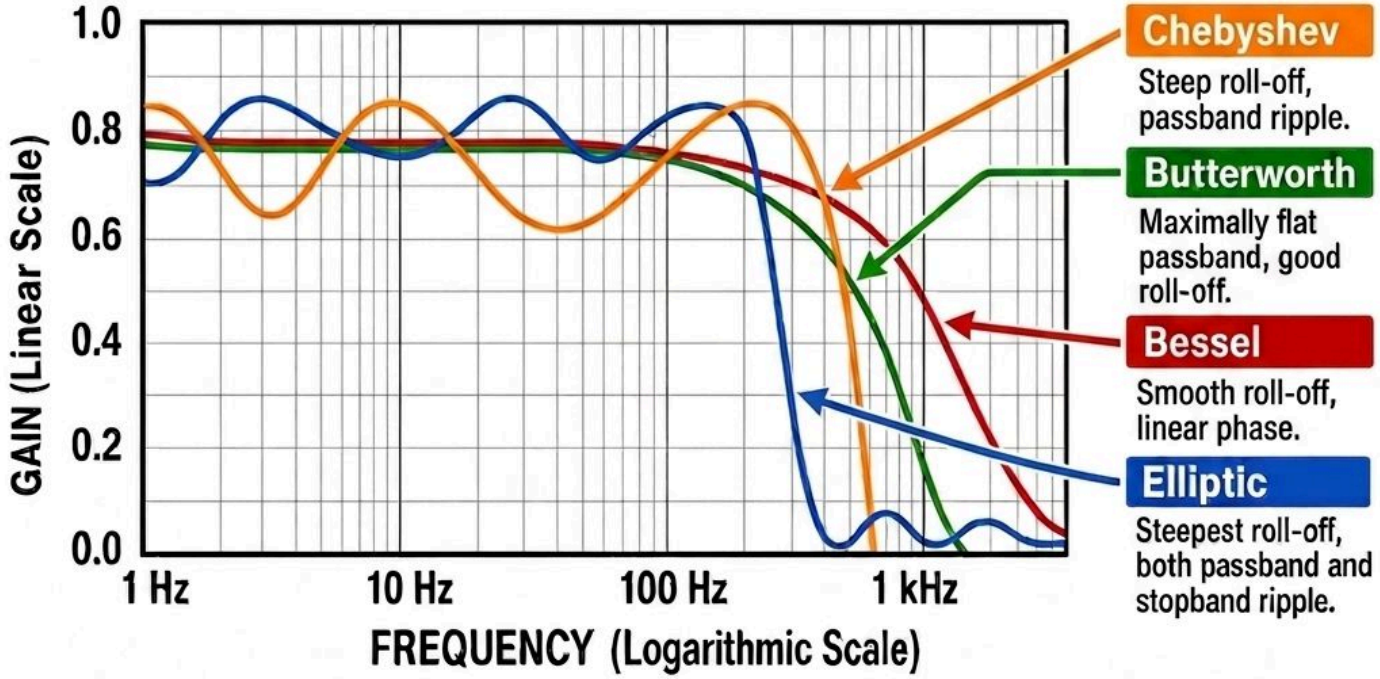
Comparación de ventanas

Ventana	Ripple	Atenuación
Rectangular	Alto	Baja
Hann	Medio	Media
Hamming	Bajo	Alta
Blackman	Muy bajo	Muy alta



Filtros IIR


Time-Domain Input Signal
(Combined Components)
LOW PASS
FILTER
Time-Domain Output Signal
(Filtered)
High-Frequency
Component
Conceptual Diagram
of Idealized Filtering
Low-Frequency
Component Passed


1.0
0.8
0.6
0.4
0.2
0.0
GAIN (Linear Scale)
1 Hz
10 Hz
100 Hz
1 kHz
FREQUENCY (Logarithmic Scale)
Chebyshev
Steep roll-off, passband ripple.
Butterworth
Maximally flat passband, good roll-off.
Bessel
Smooth roll-off, linear phase.
Elliptic
Steepest roll-off, both passband and stopband ripple.

Ecuación IIR

$$y[n] = \sum_{k=0}^{M} b_k x[n-k] - \sum_{k=1}^{N} a_k y[n-k]$$

Infinite Impulse Response

Características:

- Recursivos
- Feedback
- Polos y ceros
- Menor orden

Ventajas IIR:

- Menor orden
- Más eficientes
- Menor memoria
- Diseño desde filtros analógicos

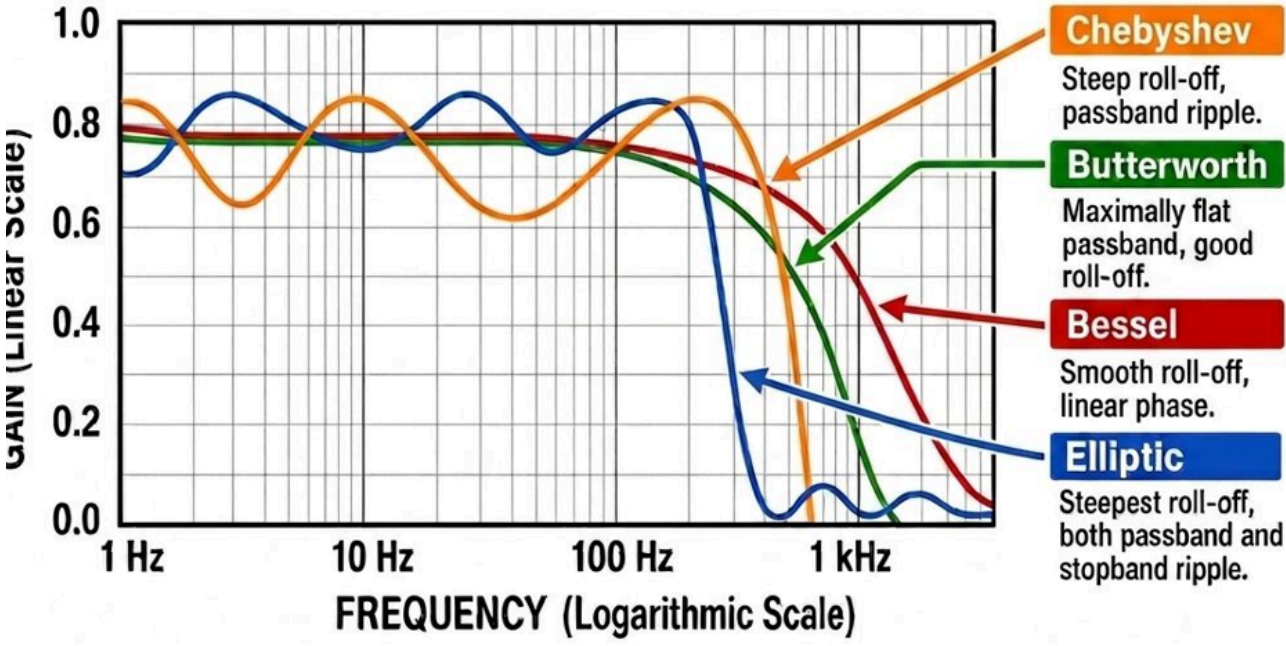
Desventajas IIR

- Posible inestabilidad
- Fase no lineal
- Más sensibles numéricamente

Familias IIR

Principales tipos

- Butterworth
- Chebyshev I
- Chebyshev II
- Elíptico
- Bessel



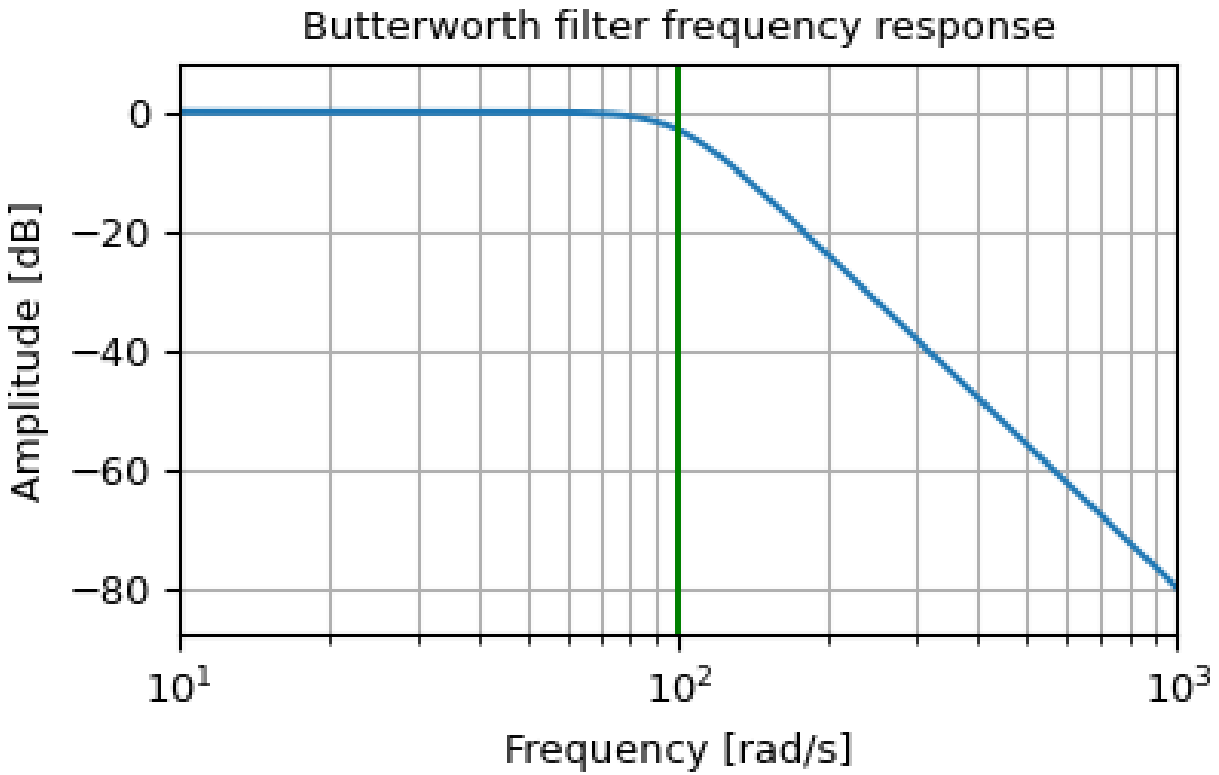
Butterworth

Características

- Máximamente plano
- Sin ripple
- Respuesta suave

```
b,a = butter(N,
    Wn,   # puede ser par
    btype='low',
    output='ba',
    fs=fs)
```

```
y = lfilter(b,a,x)
y = filtfilt(b,a,x)
```



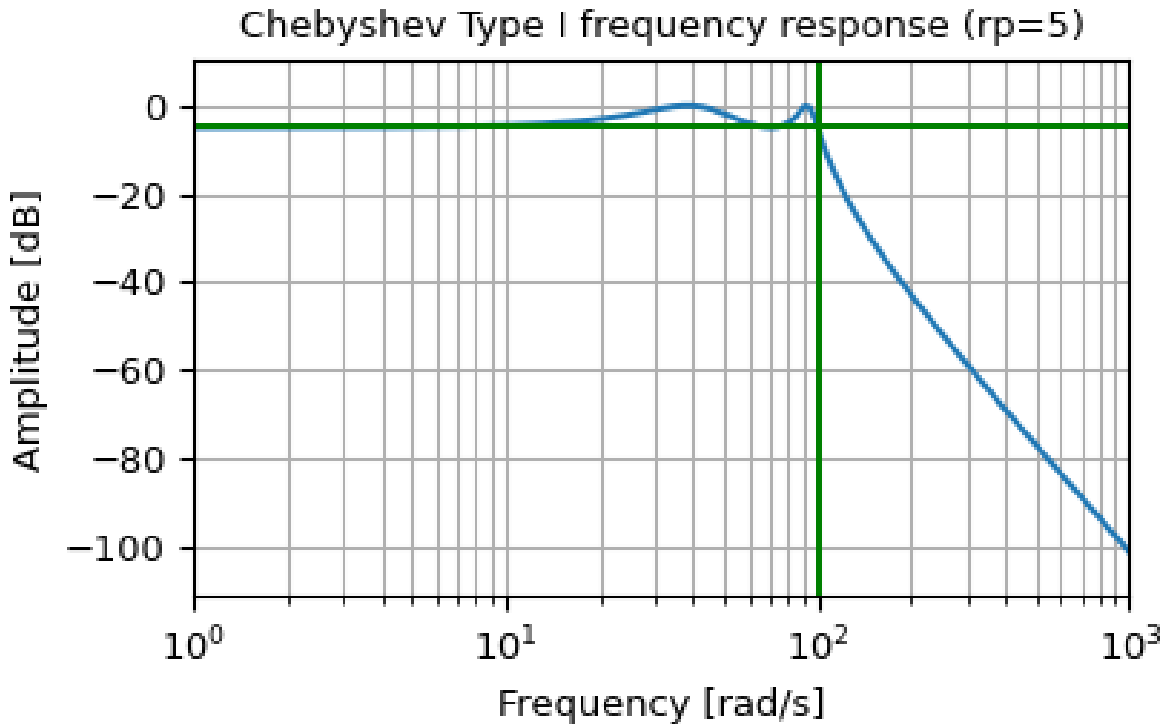
Chebyshev Tipo I

Características

- Ripple en banda pasante
- Transición abrupta
- Menor orden

```
b,a = cheby1(N,
    rp, # ripple en db, rp=5 -> -5db
    Wn, # fc en unidades de fs
    btype='low',
    output='ba',
    fs=fs)
```

```
y = lfilter(b,a,x)
y = filtfilt(b,a,x)
```



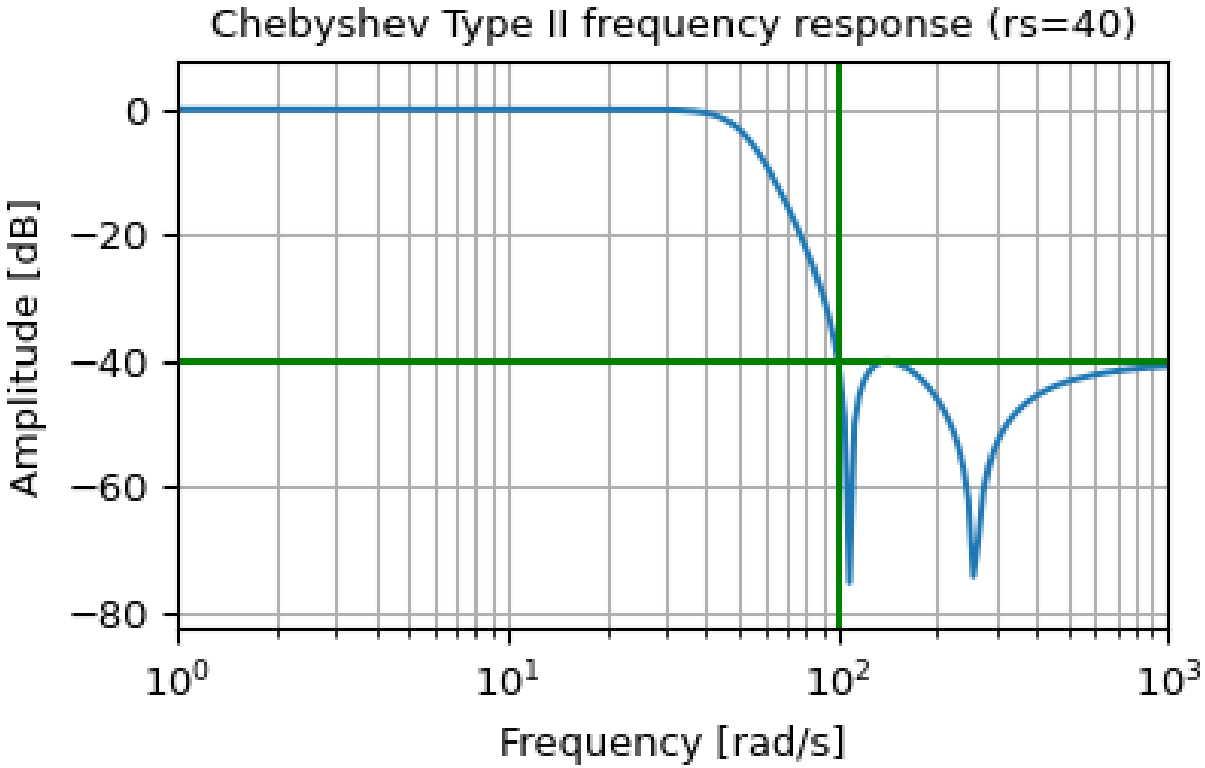
Chebyshev Tipo II

Características

- Banda pasante plana
- Ripple en rechazo

```
b,a = cheby2(N,
    rp, # ripple en db, rp=5 -> -5db
    Wn, # fc en unidades de fs
    btype='low',
    output='ba',
    fs=fs)
```

```
y = lfilter(b,a,x)
y = filtfilt(b,a,x)
```



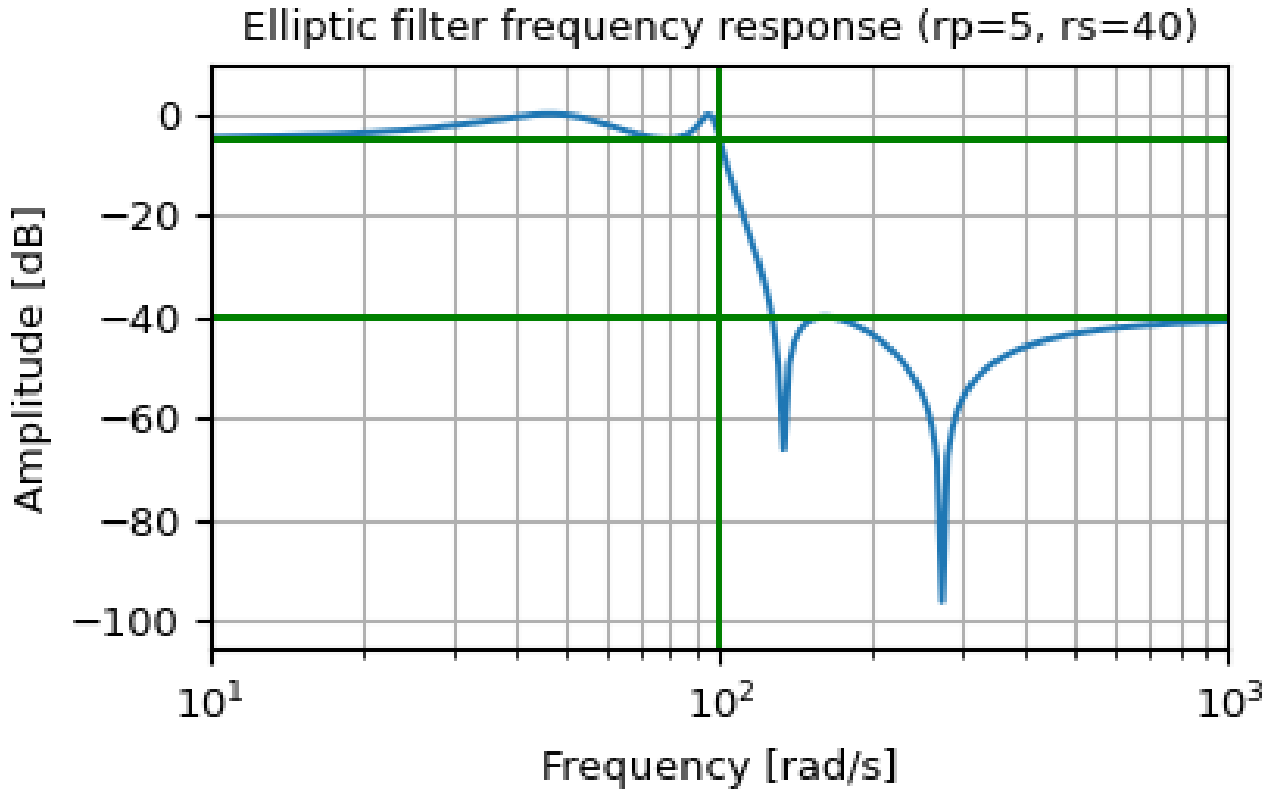
Filtros Elípticos

Características

- Ripple en ambas bandas
- Máxima selectividad
- Orden mínimo

```
b,a = ellip(N=4, #orden
    rp=5, # ripple en db, rp=5 -> -5db
    Wn=40, # rad/s
    btype='low',
    output='ba',
    analog=True)
```

```
y = lfilter(b,a,x)
y = filtfilt(b,a,x)
```



Comparación IIR

Tipo	Ripple Passband	Ripple Stopband
Butterworth	No	No
Chebyshev I	Sí	No
Chebyshev II	No	Sí
Elíptico	Sí	Sí

Respuesta de filtros IIR en python

```
# Butterworth High-pass @ 200 Hz
b_hp, a_hp = butter(order_iir, 200/nyq, btype='highpass', output='ba')

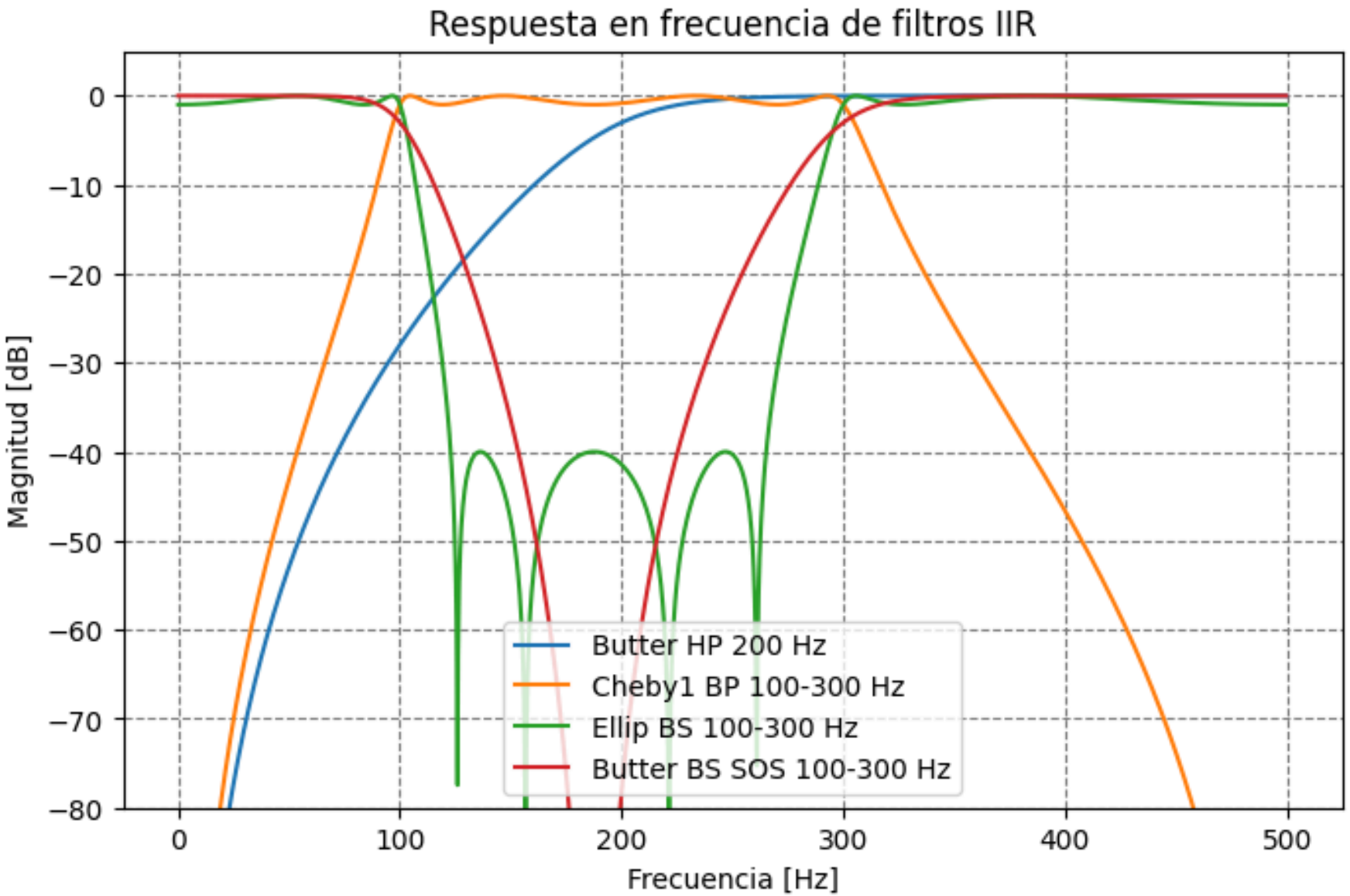
# Chebyshev I Band-pass 100-300 Hz (ripple 1 dB)
b_bp, a_bp = cheby1(order_iir, 1, [100/nyq, 300/nyq], btype='bandpass', output='ba')

# Elíptico Band-stop 100-300 Hz (ripple 1 dB, atenuación de rechazo 40 dB)
b_bs, a_bs = ellip(order_iir, 1, 40, [100/nyq, 300/nyq], btype='bandstop', output='ba')

# SOS (Second-Order Sections) para mayor estabilidad - Butterworth Band-stop
sos_bs = iirfilter(order_iir, [100/nyq, 300/nyq], btype='bandstop', ftype='butter', output='sos')

# Respuestas en frecuencia
w_hp, h_hp = freqz(b_hp, a_hp, worN=2048, fs=fs)
w_bp, h_bp = freqz(b_bp, a_bp, worN=2048, fs=fs)
w_bs, h_bs = freqz(b_bs, a_bs, worN=2048, fs=fs)
w_sos, h_sos = sosfreqz(sos_bs, worN=2048, fs=fs)

# Gráfica
plt.plot(w_hp, 20*np.log10(np.abs(h_hp)), label='Butter HP 200 Hz')
plt.plot(w_bp, 20*np.log10(np.abs(h_bp)), label='Cheby1 BP 100-300 Hz')
plt.plot(w_bs, 20*np.log10(np.abs(h_bs)), label='Ellip BS 100-300 Hz')
plt.plot(w_sos, 20*np.log10(np.abs(h_sos)), label='Butter BS SOS 100-300 Hz')
plt.title('Respuesta en frecuencia de filtros IIR')
plt.xlabel('Frecuencia [Hz]')
plt.ylabel('Magnitud [dB]')
plt.ylim(-80, 5)
plt.legend()
plt.grid(True)
plt.show()
```

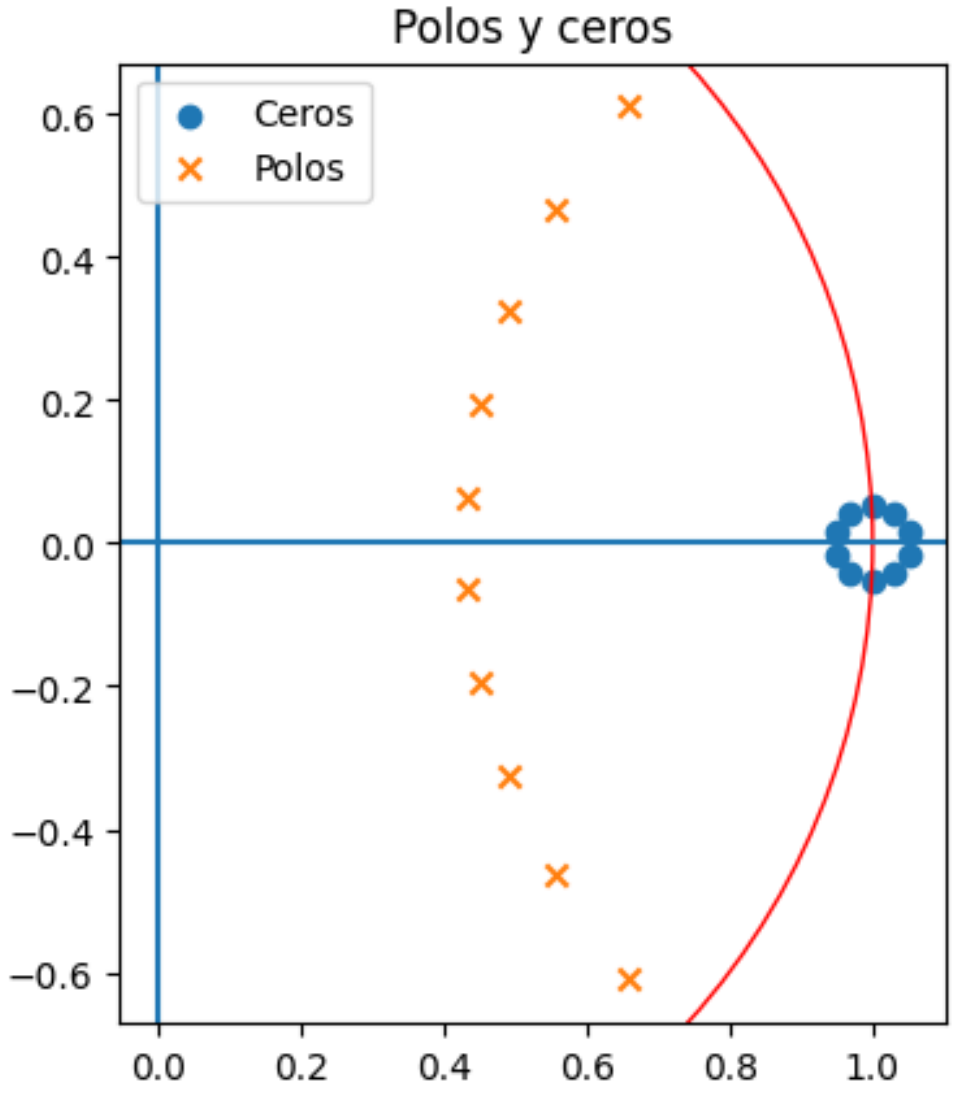

Respuesta en frecuencia de filtros IIR
Magnitud [dB]
0
−10
−20
−30
−40
−50
−60
−70
−80
0
100
200
300
400
500
Frecuencia [Hz]
Butter HP 200 Hz
Cheby1 BP 100-300 Hz
Ellip BS 100-300 Hz
Butter BS SOS 100-300 Hz

Estabilidad

Condición

$$|p_k| < 1$$

Todos los polos deben estar dentro del círculo unidad.



Biquads

Secciones de segundo orden

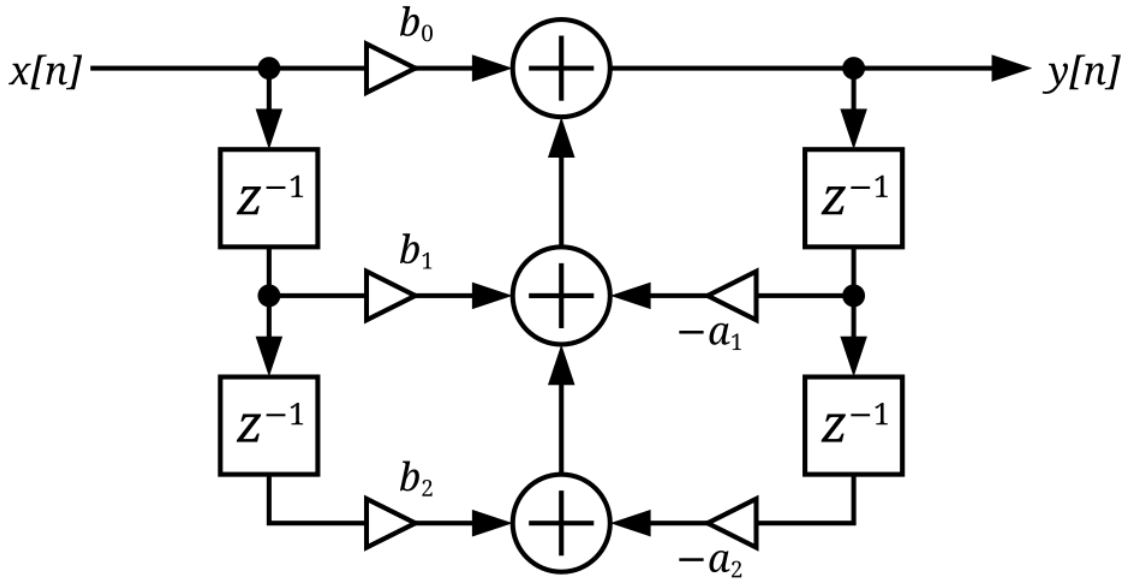
$$H(z) = \frac{b_0 + b_1 z^{-1} + b_2 z^{-2}}{1 + a_1 z^{-1} + a_2 z^{-2}}$$

Direct Form I

- Más memoria

Direct Form II

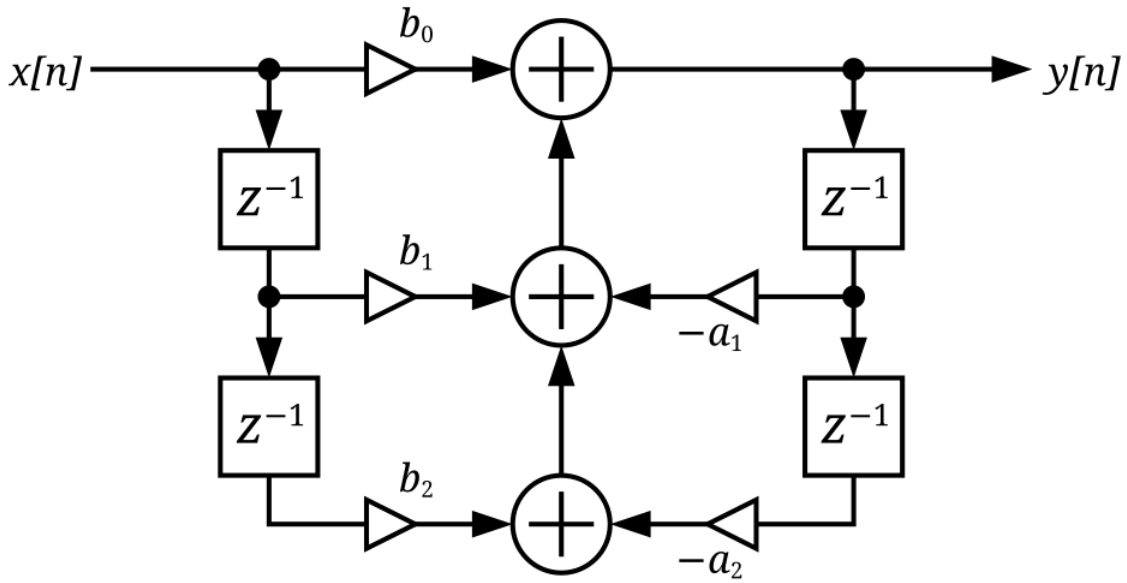
- Menor memoria



¿Por qué usar biquads?

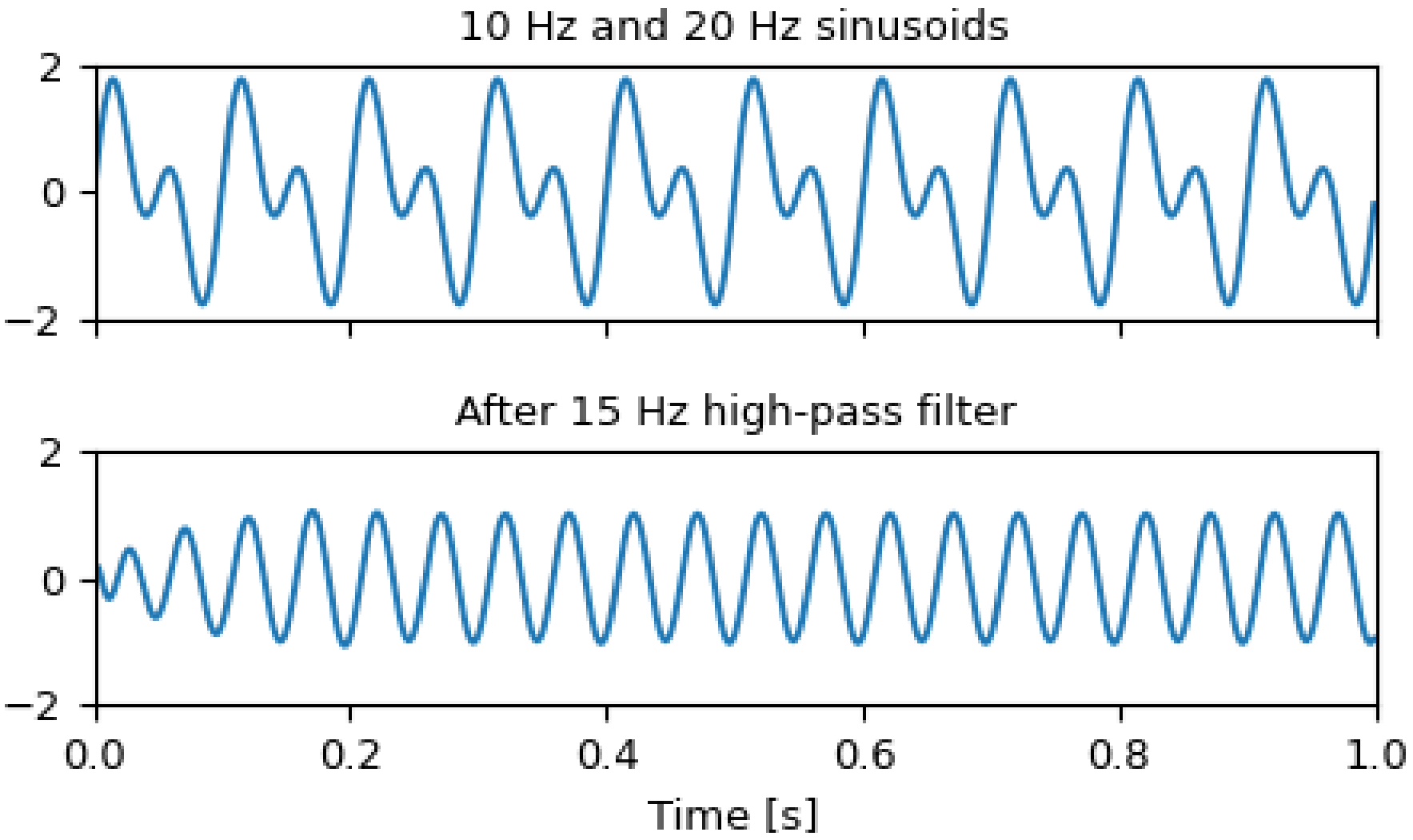
Ventajas

- Mejor estabilidad numérica
- Implementación robusta
- Uso industrial estándar
- Ideal para DSP embebido



Biquads en python

```
sos = signal.butter(10, 15, 'hp', fs=1000, output='sos')
filtered = signal.sosfilt(sos, sig)
ax2.plot(t, filtered)
ax2.set_title('After 15 Hz high-pass filter')
ax2.axis([0, 1, -2, 2])
ax2.set_xlabel('Time [s]')
plt.tight_layout()
plt.show()
```


10 Hz and 20 Hz sinusoids
2
0
−2
After 15 Hz high-pass filter
2
0
−2
0.0
0.2
0.4
0.6
0.8
1.0
Time [s]